ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

NextRAN-AI 29/10/2026

Integrated Systems Laboratory (ETH Zürich)

Marco Bertuletti mbertuletti@iis.ee.ethz.ch
Yichao Zhang yiczhang@iis.ee.ethz.ch
Mahdi Abdollahpour mahdi.abdollahpout@unibo.it
Paolo Galfano pgalfano@iis.ee.ethz.ch
Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch
Luca Benini lbenini@iis.ee.ethz.ch



pulp-platform.org
@pulp_platform
company/pulp-platform
youtube.com/pulp_platform

PULP Platform
Open Source Hardware, the way it should be!

Agenda

- **Optimizations on the matrix-compute-engine and interconnect**
- **Code optimizations on the L1 Transformer**
- **Summary of completed 3D-IC folding exploration**
  - Group-on-Group (GoG)
  - Compute-on-Interconnect (CoI) (new)

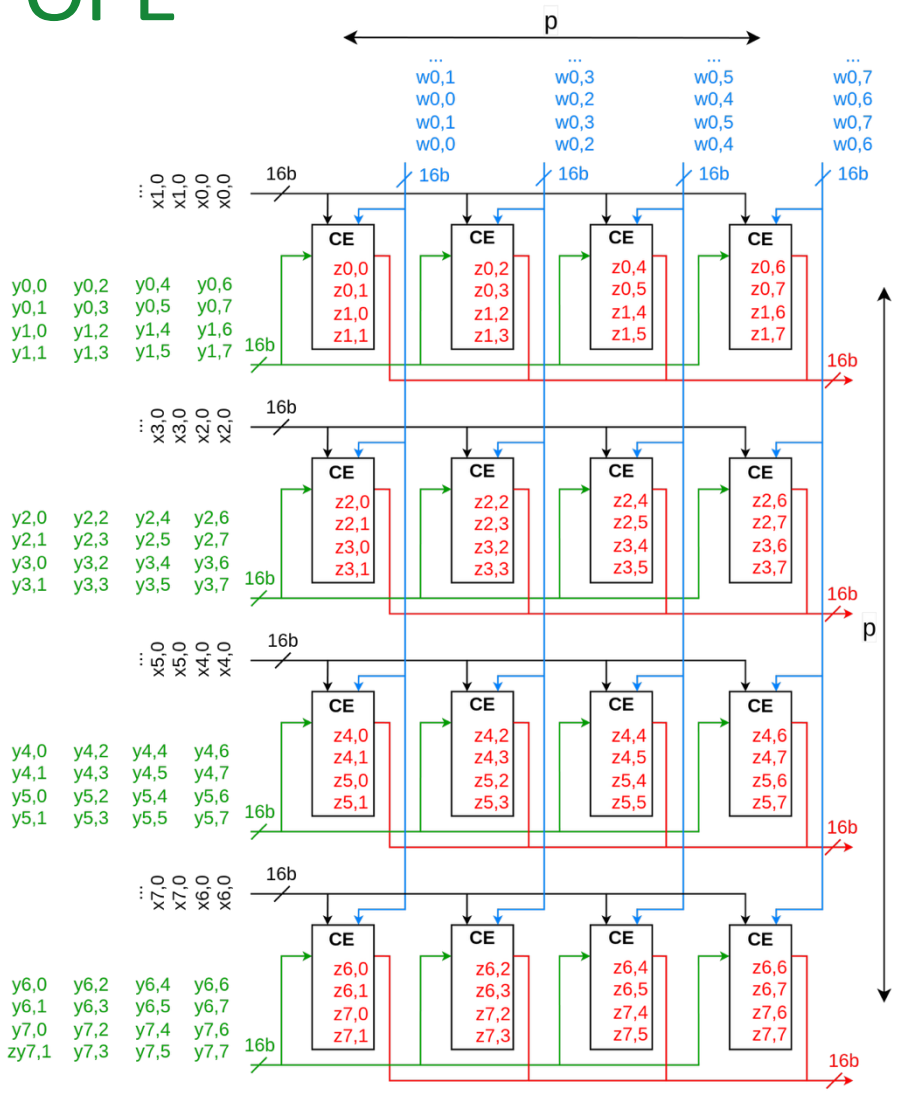
Optimizations on the Engine & interconnect

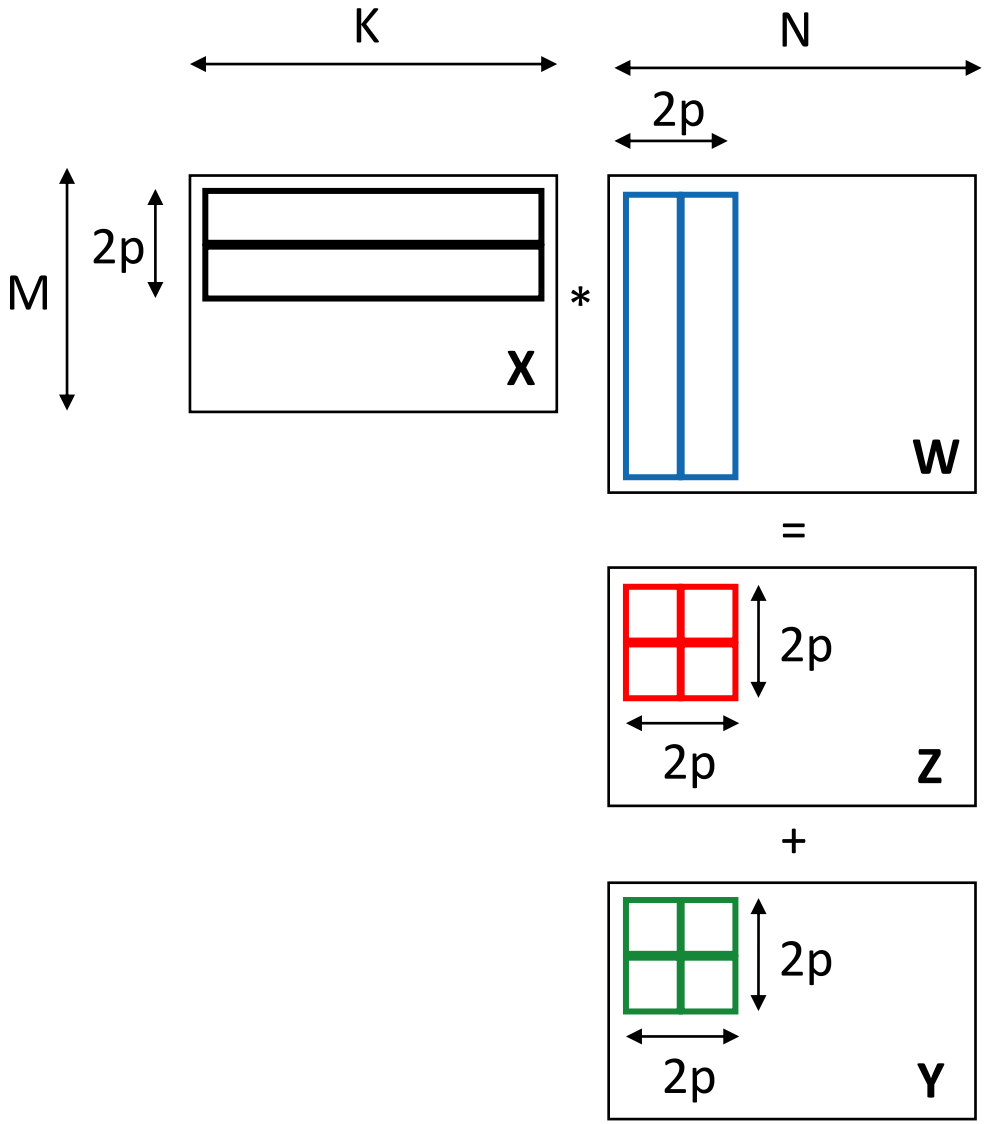


Planned actions (25/09/26):

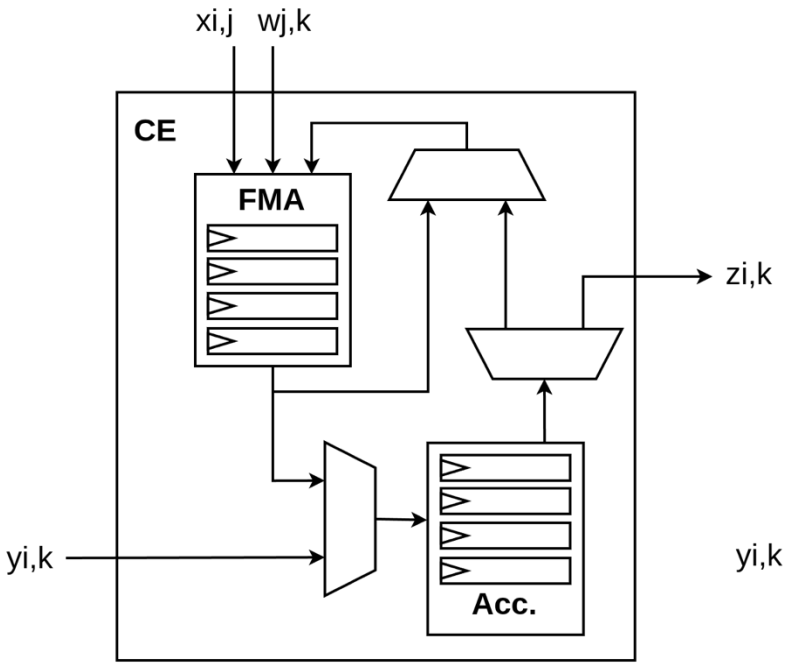
- Experiment the outer-product engine
- Examine Bottlenecks for small problem sizes

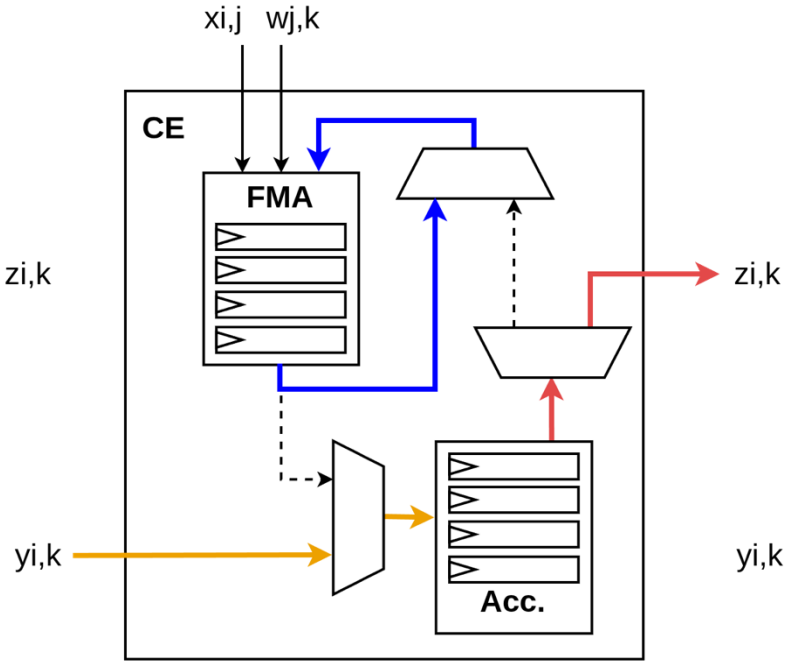
O-POPE


p
w0,1 w0,0 w0,1 w0,0
w0,3 w0,2 w0,3 w0,2
w0,5 w0,4 w0,5 w0,4
w0,7 w0,6 w0,7 w0,6
16b
CE
z0,0 z0,1 z1,0 z1,1
z0,2 z0,3 z1,2 z1,3
z0,4 z0,5 z1,4 z1,5
z0,6 z0,7 z1,6 z1,7
z2,0 z2,1 z3,0 z3,1
z2,2 z2,3 z3,2 z3,3
z2,4 z2,5 z3,4 z3,5
z2,6 z2,7 z3,6 z3,7
z4,0 z4,1 z5,0 z5,1
z4,2 z4,3 z5,2 z5,3
z4,4 z4,5 z5,4 z5,5
z4,6 z4,7 z5,6 z5,7
z6,0 z6,1 z7,0 z7,1
z6,2 z6,3 z7,2 z7,3
z6,4 z6,5 z7,4 z7,5
z6,6 z6,7 z7,6 z7,7
x1,0 x1,0 x0,0 x0,0
x3,0 x3,0 x2,0 x2,0
x5,0 x5,0 x4,0 x4,0
x7,0 x7,0 x6,0 x6,0
y0,0 y0,2 y0,4 y0,6
y0,1 y0,3 y0,5 y0,7
y1,0 y1,2 y1,4 y1,6
y1,1 y1,3 y1,5 y1,7
y2,0 y2,2 y2,4 y2,6
y2,1 y2,3 y2,5 y2,7
y3,0 y3,2 y3,4 y3,6
y3,1 y3,3 y3,5 y3,7
y4,0 y4,2 y4,4 y4,6
y4,1 y4,3 y4,5 y4,7
y5,0 y5,2 y5,4 y5,6
y5,1 y5,3 y5,5 y5,7
y6,0 y6,2 y6,4 y6,6
y6,1 y6,3 y6,5 y6,7
y7,0 y7,2 y7,4 y7,6
zy7,1 y7,3 y7,5 y7,7
p

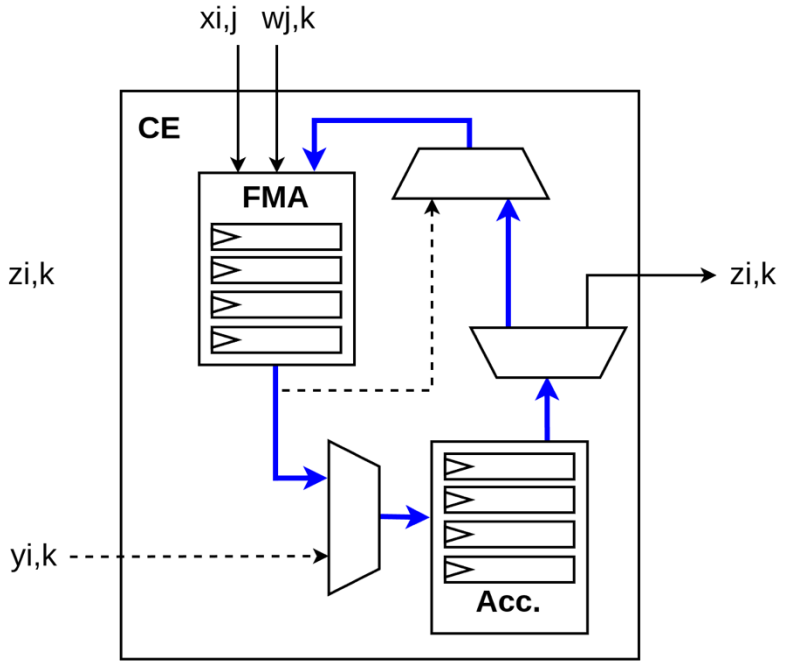

K
N
2p
M
2p
X
*
W
=
2p
2p
Z
+
2p
2p
Y

O-POPE





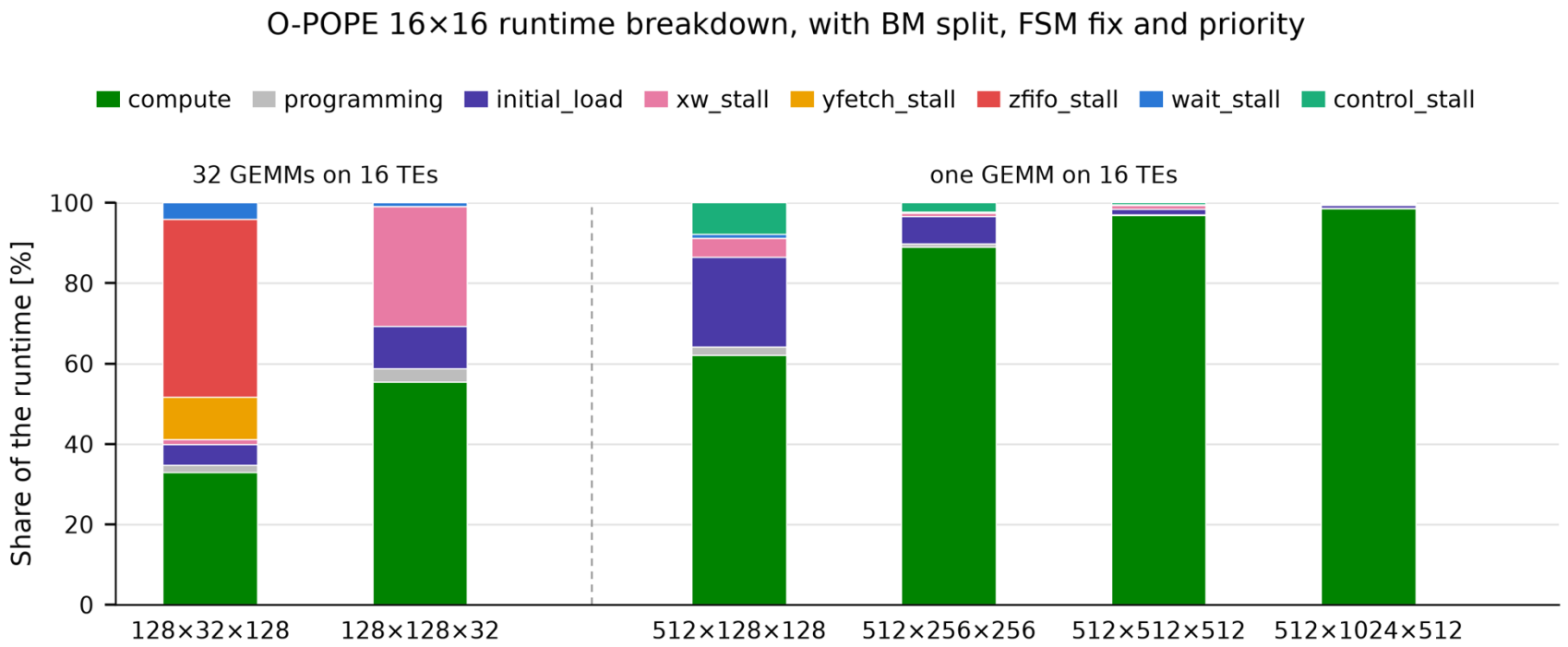
During the inner-loop:
1. Partial results loop into the array
2. Previous output is read from accumulators
3. Accumulators are prefilled with the next Ys



When we change output-tile:
1. Empty the Accumulators into the array FMAs
& Empty the array FMAs into the accumulators

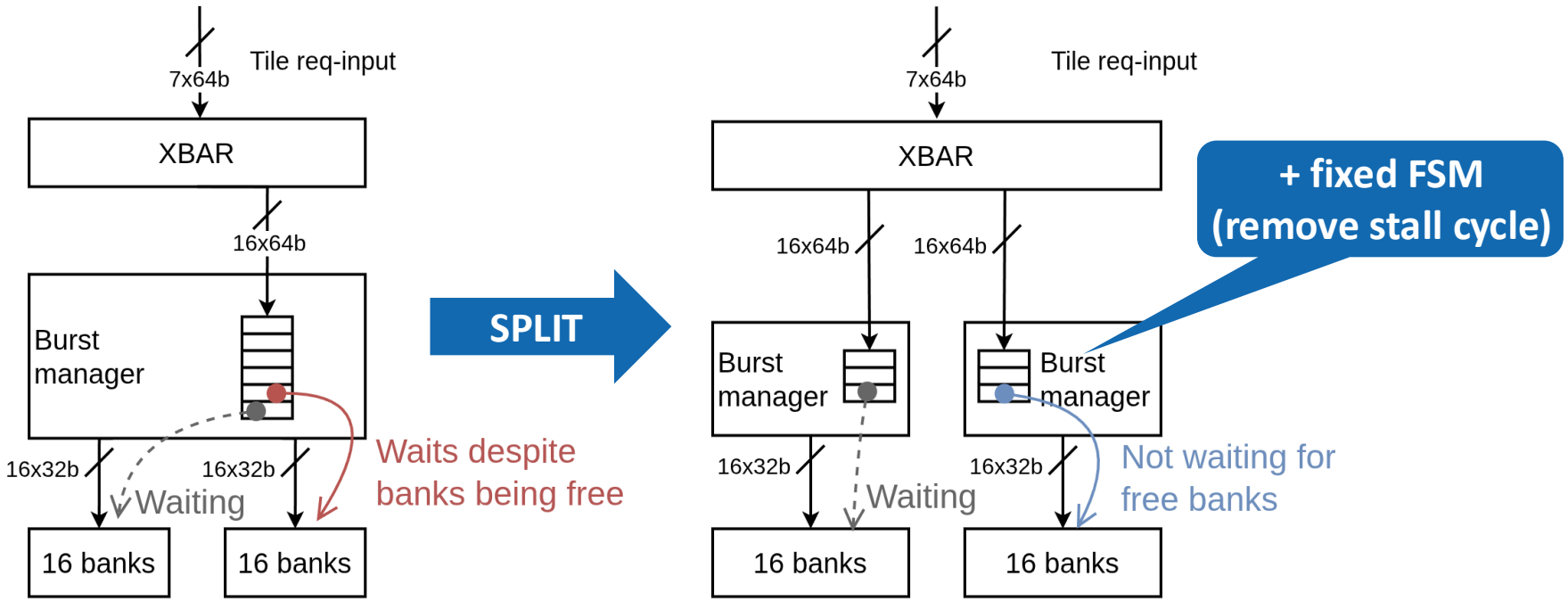
The Main Limiter of Utilization

- Small matrices: initial Y bias loading limits utilization
- For small inner dimension K: next-tile Y fetching and Z output storing



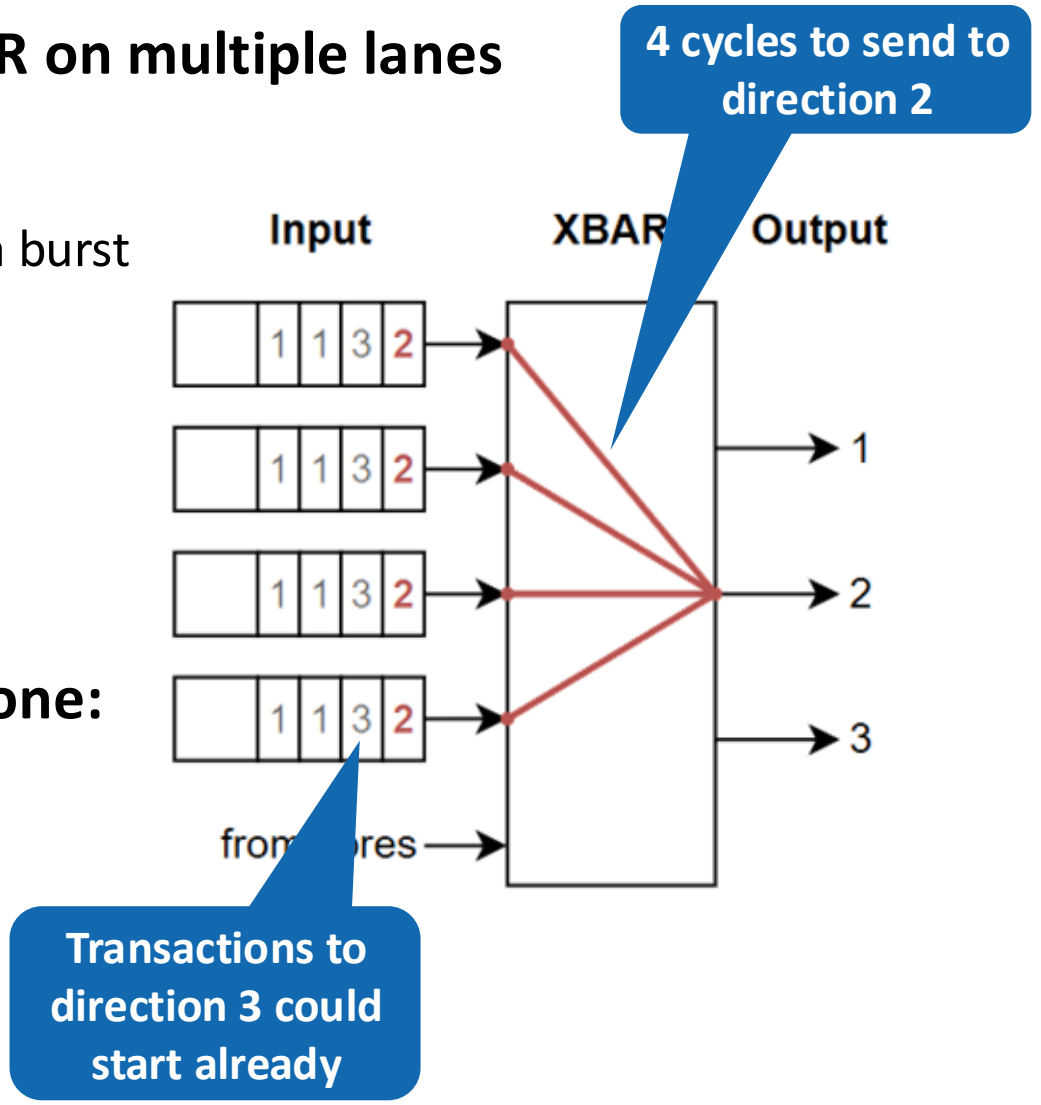
Performance improvements from small enhancements

- **With a single burst manager for all tile's banks**
  - A request targeting free banks may wait behind a stalled request targeting busy banks
  - Banks are kept busy while waiting for responses to get accepted in the remote interconnect
- **Fixed O-POPE's priority scheme giving Y higher priority and X/W equal priority**



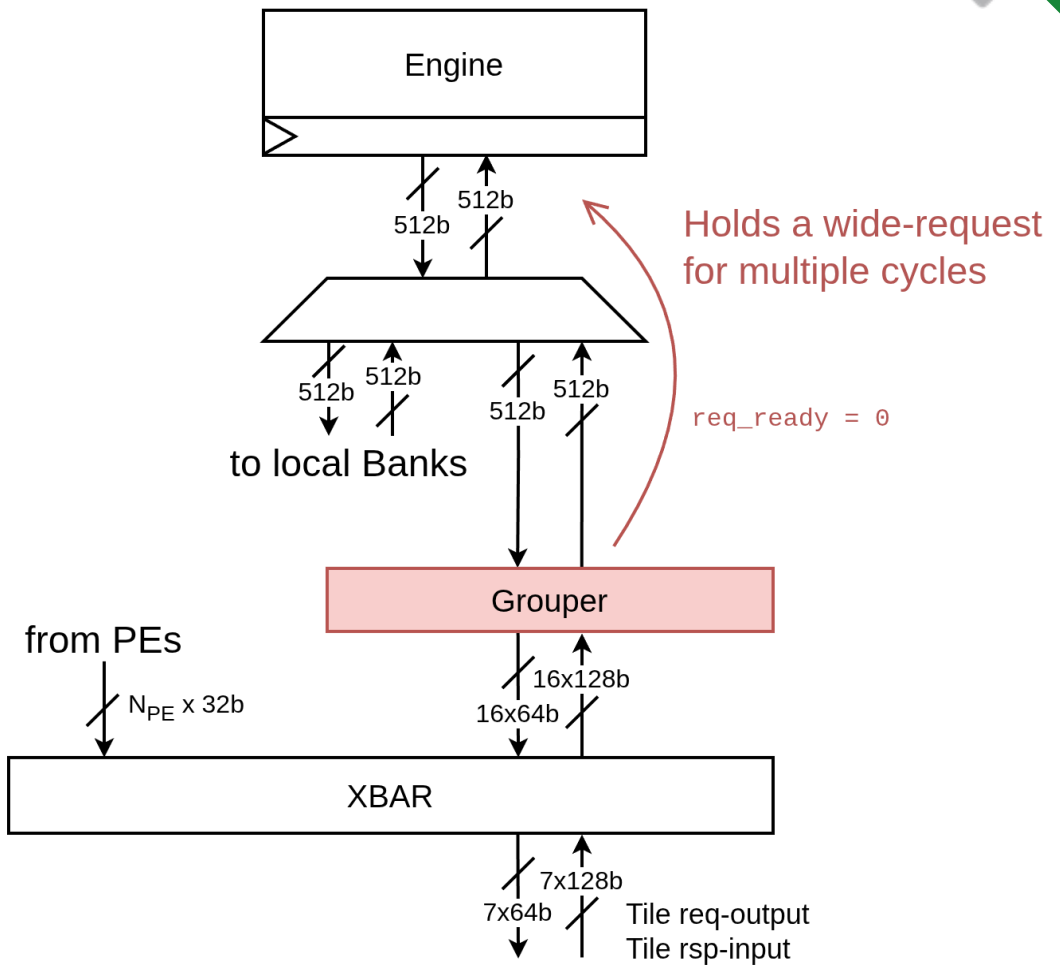
Major Bottleneck: Head-of-Line Blocking

- **Bursted requests/responses reach the XBAR on multiple lanes**
- **Assuming target direction is free:**
  - It takes multiple cycles to send all transactions of a burst
- **If the direction is not free:**
  - The transactions wait additional cycles
- **Transactions that require a different, free direction are blocked behind the previous one: <u>Head-of-Line blocking</u> (HOL)**



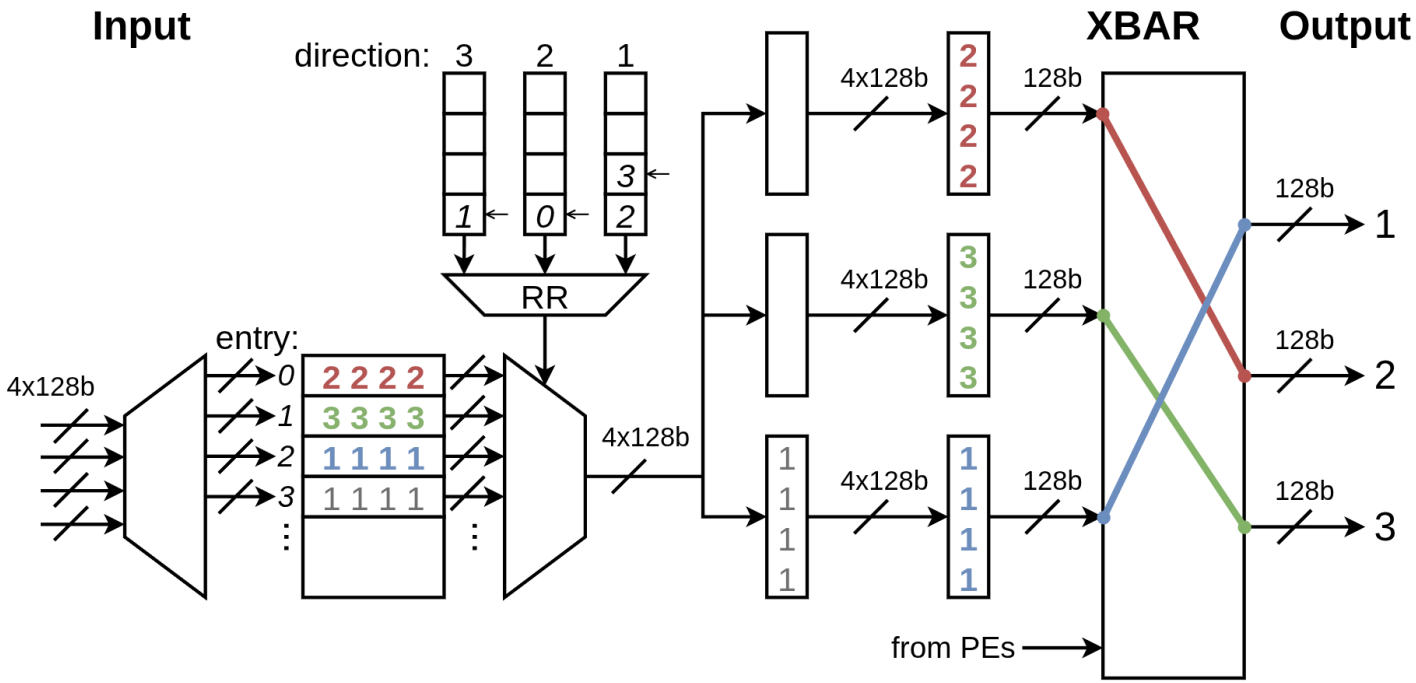
Initiator-Side Request Path

- **A 512-bit requests requires 8 cycles to cross the XBAR**
  - It holds the engine port until all 8 64-bit wide requests have been sent
- **Output is composed of $4p^2$ elements, i.e. $2p$ 512-bit wide words**
  - With 8 cycles per 512-bit word it takes 256 cycles
  - On thin inner dimension ($K = 32$), the inner loop completes is $4K = 128$ cycles.
- **This structural bottleneck makes impossible to achieve high utilization when K is small**

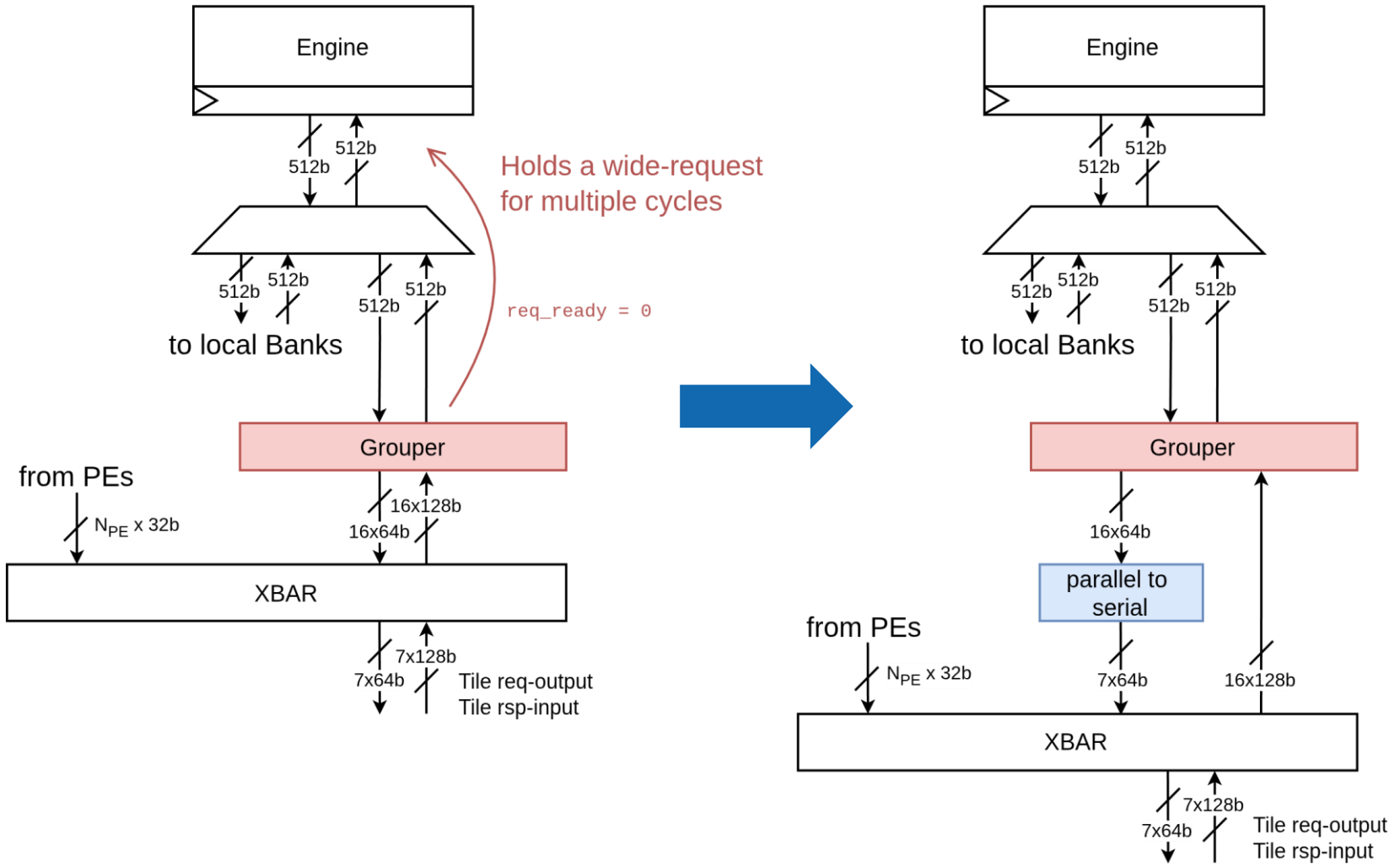


Virtual Output Queueing

- A shared buffer contains all burst transactions: single read and write ports.
- Parallel write – serial read: transactions that cross the XBAR in parallel are stored in a staging register (one per direction).

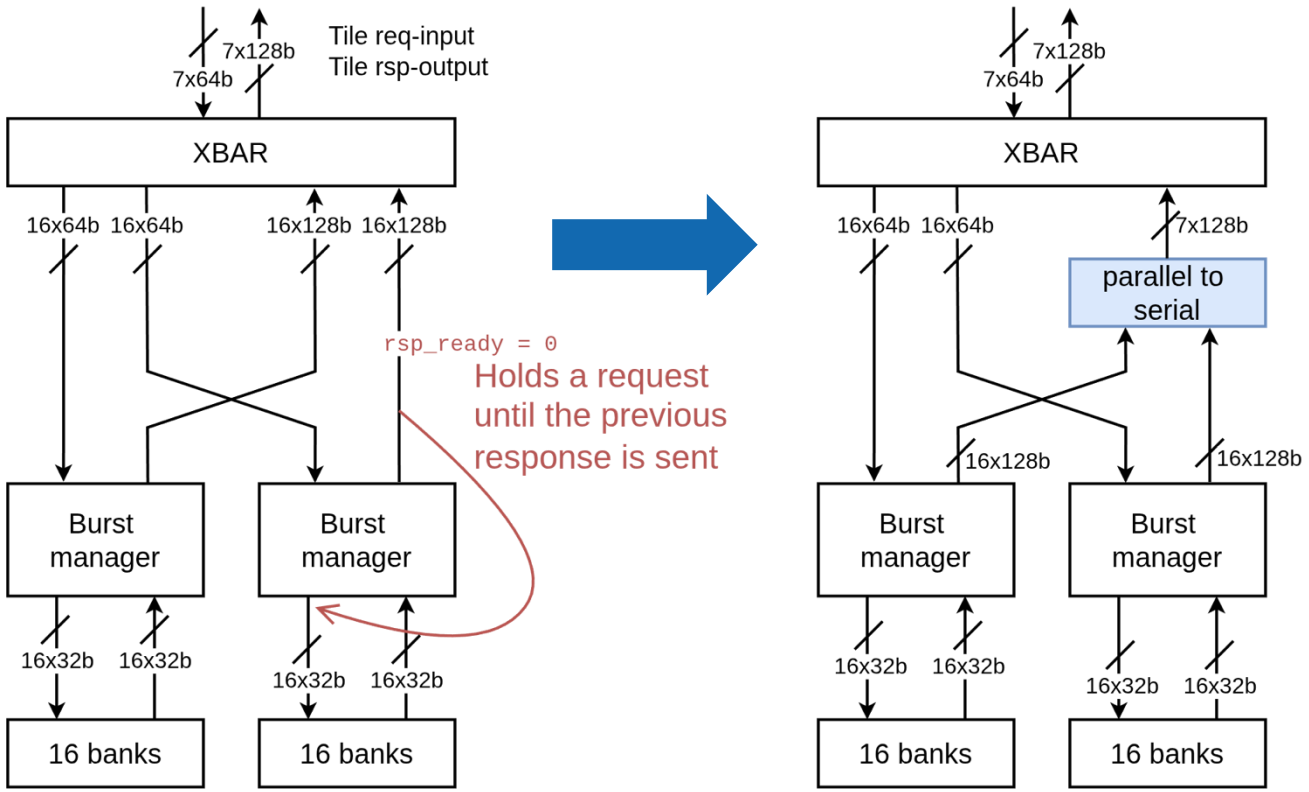


Virtual Output Queueing

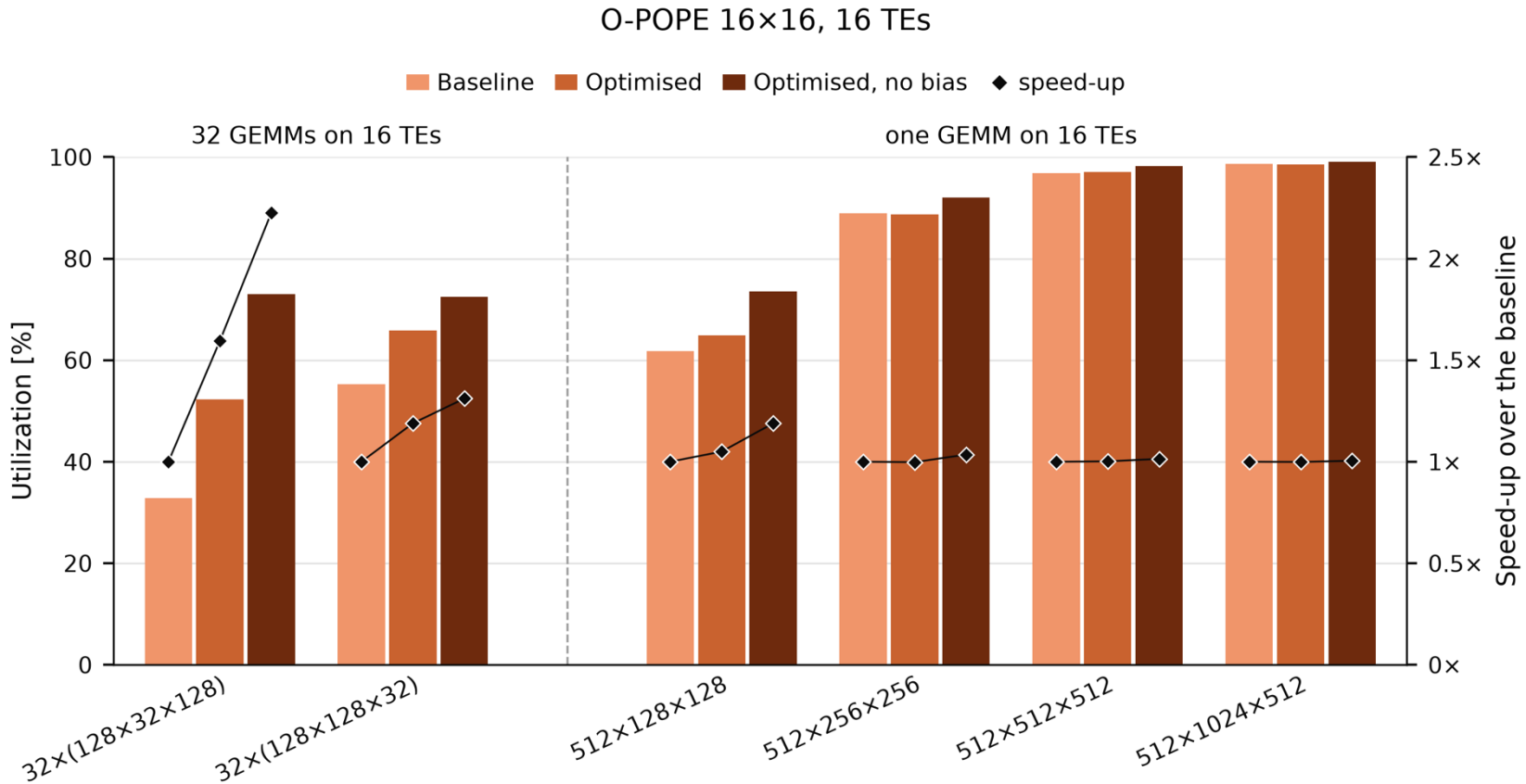


Target-Side Response Path

- The burst manager holds the banks until the response is sent
- Same issue as for the request path ➔ Can be solved with VOQ as well

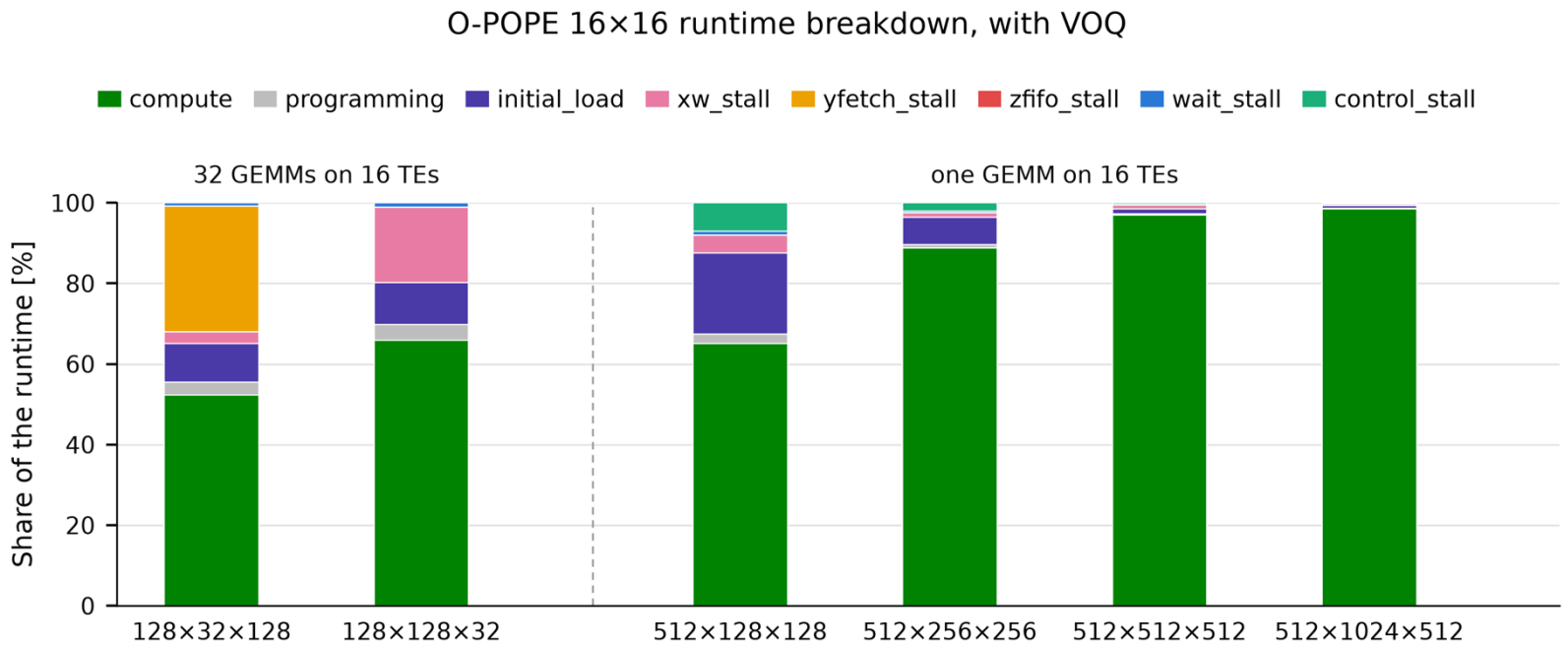


Results – Runtime and Utilization



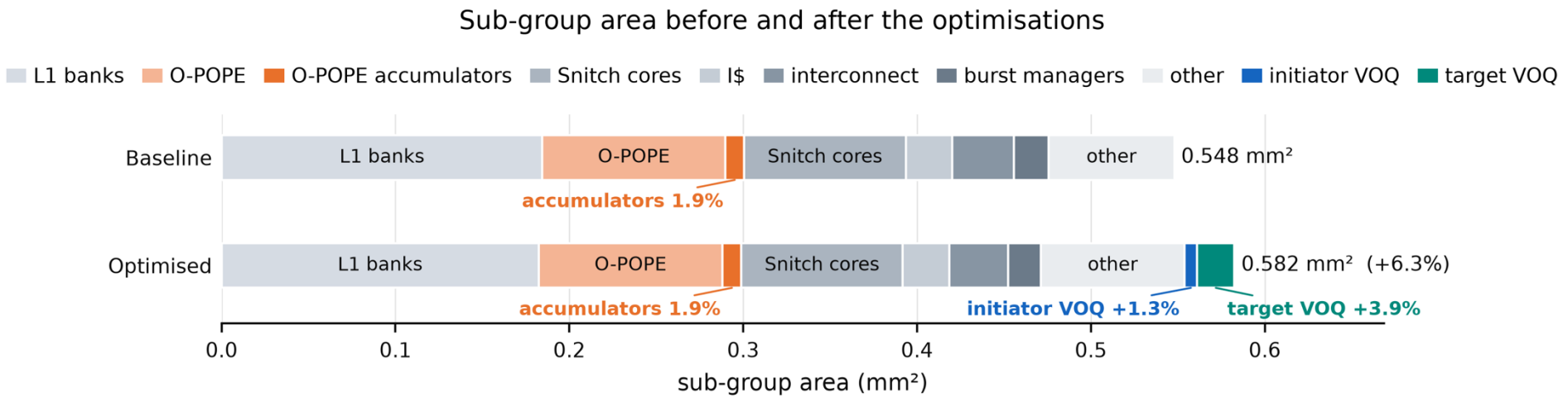
Stalls Breakdown

- **No more Z output FIFO stalls (red)**
  - Mostly Y bias loading stalls (yellow), mainly on first tile for small matrices (purple)
  - And X/W stalls (pink)

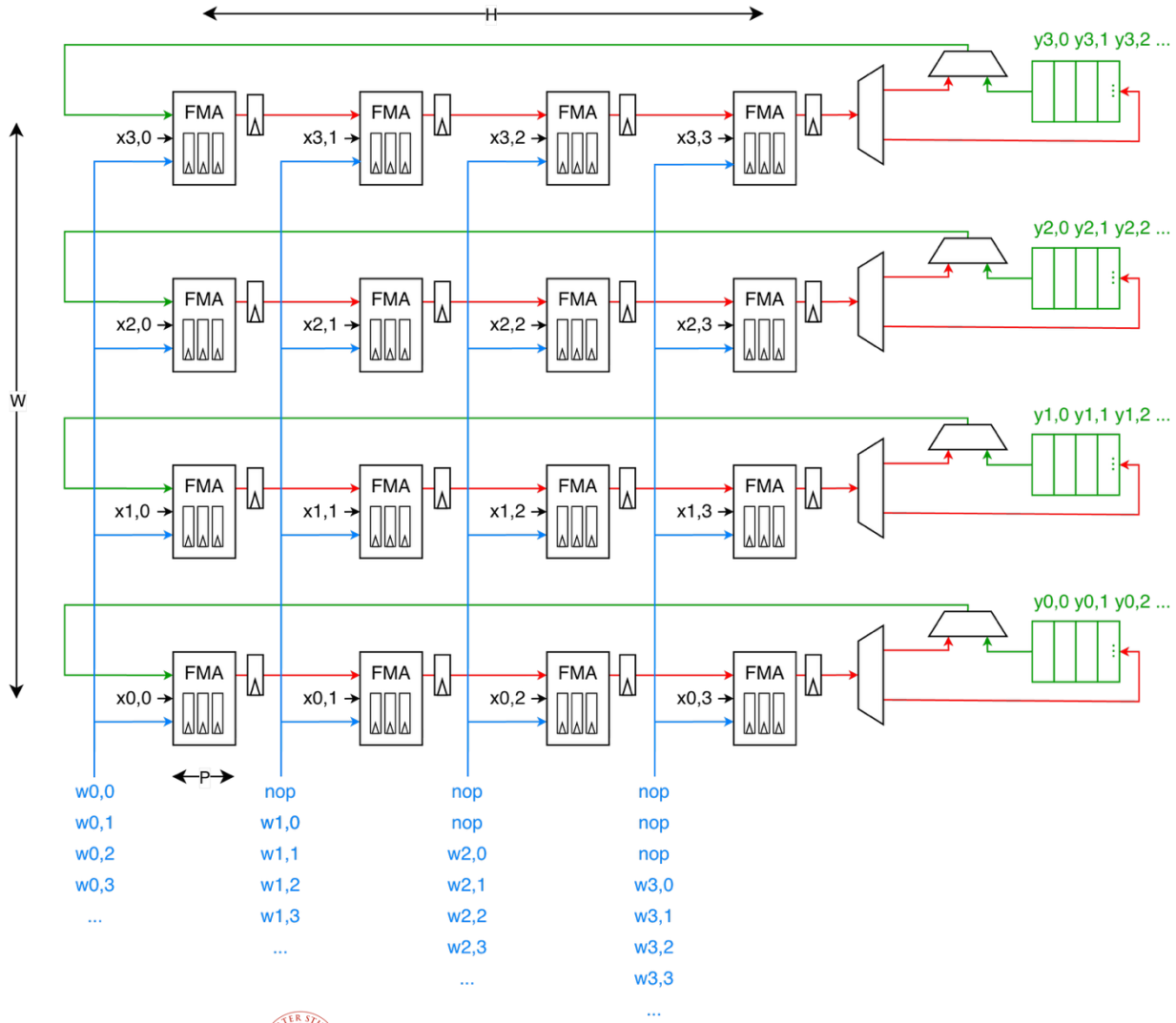


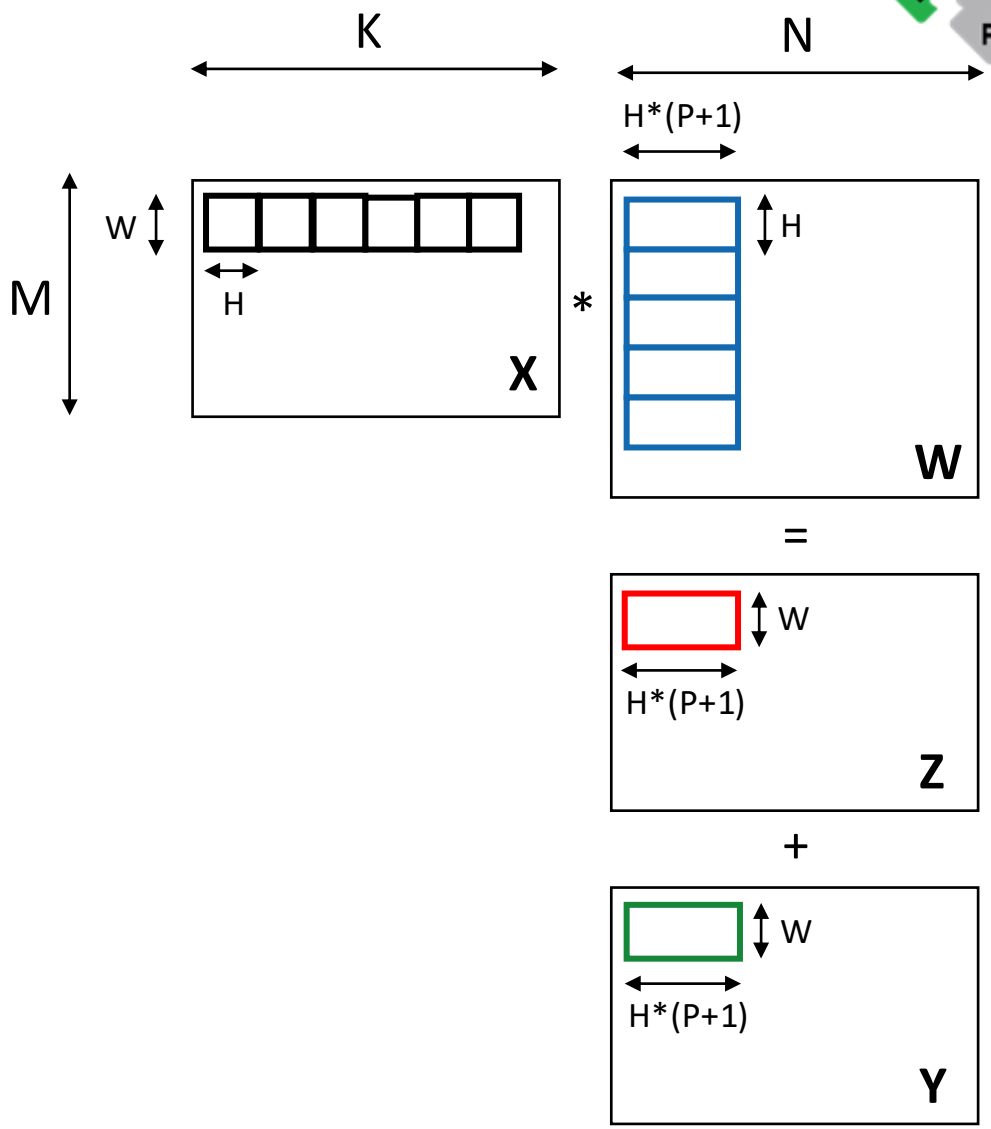
Results – Backend

- **Overall +6.3% area on the whole SubGroup**
  - Initiator-side **requests VOQ** (one per SubGroup): +1.3%
  - Target-side **response VOQ** (four per SubGroup): +3.9%

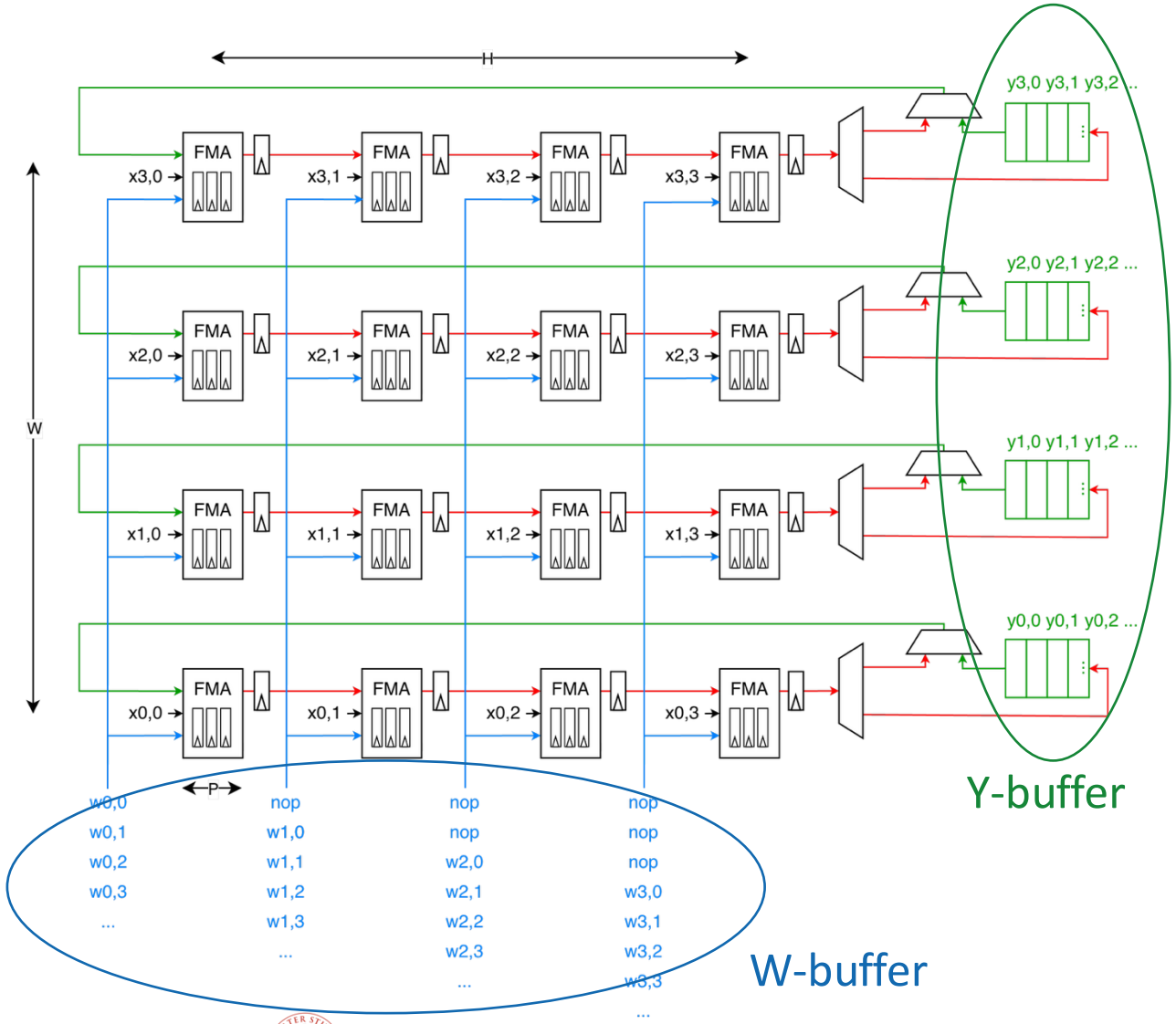


RedMulE


H
W
P
FMA
x3,0
x3,1
x3,2
x3,3
x2,0
x2,1
x2,2
x2,3
x1,0
x1,1
x1,2
x1,3
x0,0
x0,1
x0,2
x0,3
y3,0 y3,1 y3,2 ...
y2,0 y2,1 y2,2 ...
y1,0 y1,1 y1,2 ...
y0,0 y0,1 y0,2 ...
w0,0
w0,1
w0,2
w0,3
nop
w1,0
w1,1
w1,2
w1,3
w2,0
w2,1
w2,2
w2,3
w3,0
w3,1
w3,2
w3,3


K
N
H*(P+1)
M
W
H
X
*
H
W
=
W
H*(P+1)
Z
+
W
H*(P+1)
Y

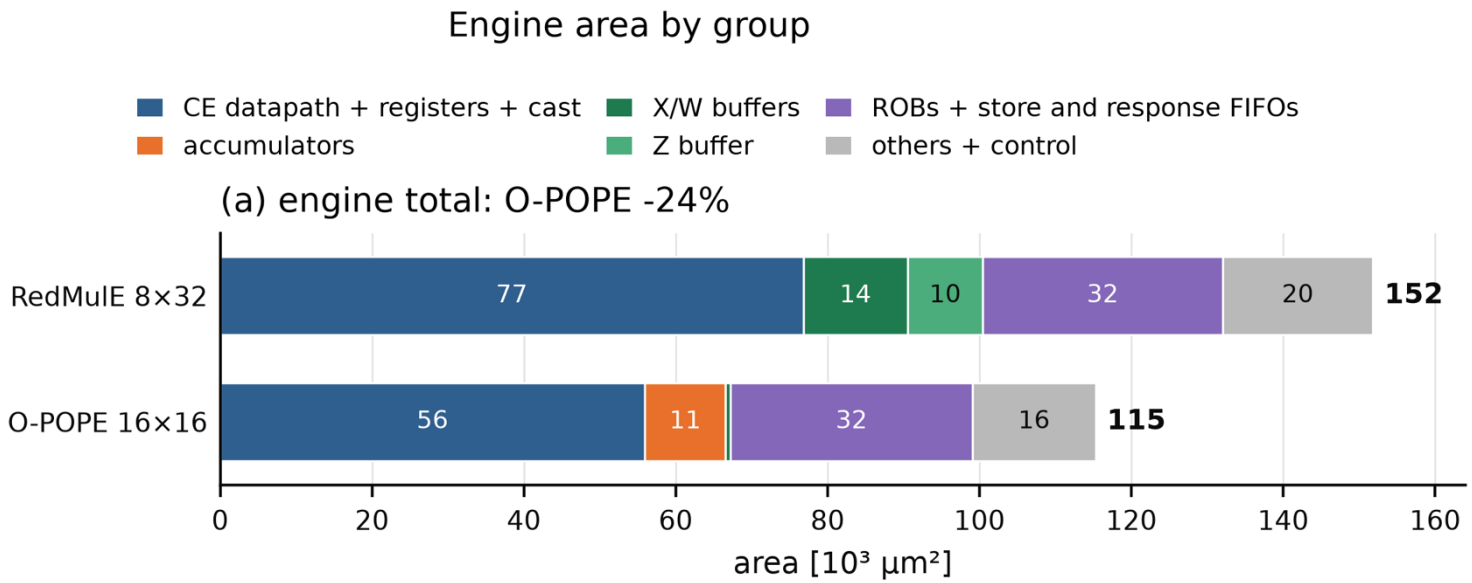
RedMulE



Requires X/W/Z-buffers O-POPE only needs The accumulators

- Each row executes inner product
- Input stationary = requires X-buffer
- Requires W-buffer ➔ loads H*(P+1) elements every (P+1) cycles
- Is output-stationary ➔ partial sums loop within the array. Requires Y-buffer to pre-load the new inputs while outputs are computed

Area Breakdown – O-POPE vs RedMulE



- Same area of ROBs+FIFOs in Streamer
- **2.2x** larger area of buffers
- **(irrelevant wrt the storage topic)** RedMulE was configured to support 8/16-bit computation, while O-POPE supports 16-bit, so area of CEs and control is larger in this backend run

Specific disadvantage of RedMulE in TensorPool

- **The latency-tolerance of the engines is governed by the number of pipeline stages in the FPUs**
  - When the FPUs are deeply pipelined the local memory accumulators & Z/Y buffers increases
  - The Streamer has more time to bring new data to the array ➔ can tolerate longer latencies!
- **In RedMulE the number of pipeline stages changes the size of the streamer memory port H*(P+1).**
- **In TensorPool wide memory accesses are serialized when they go through tile2tile interconnects, therefore their latency increases ➔ no benefit in increasing P!**

What's left to target

- **Initiator-response is a possible next candidate.**
- **Study where latency comes from and whether it can be reduced.**
- **Initial loading of Y can be mitigated with:**
  - Not loading Y if not needed (matmul)
  - Chaining of multiple GEMM jobs
  - Use more, smaller TEs

Optimizations on the L1-Transformer

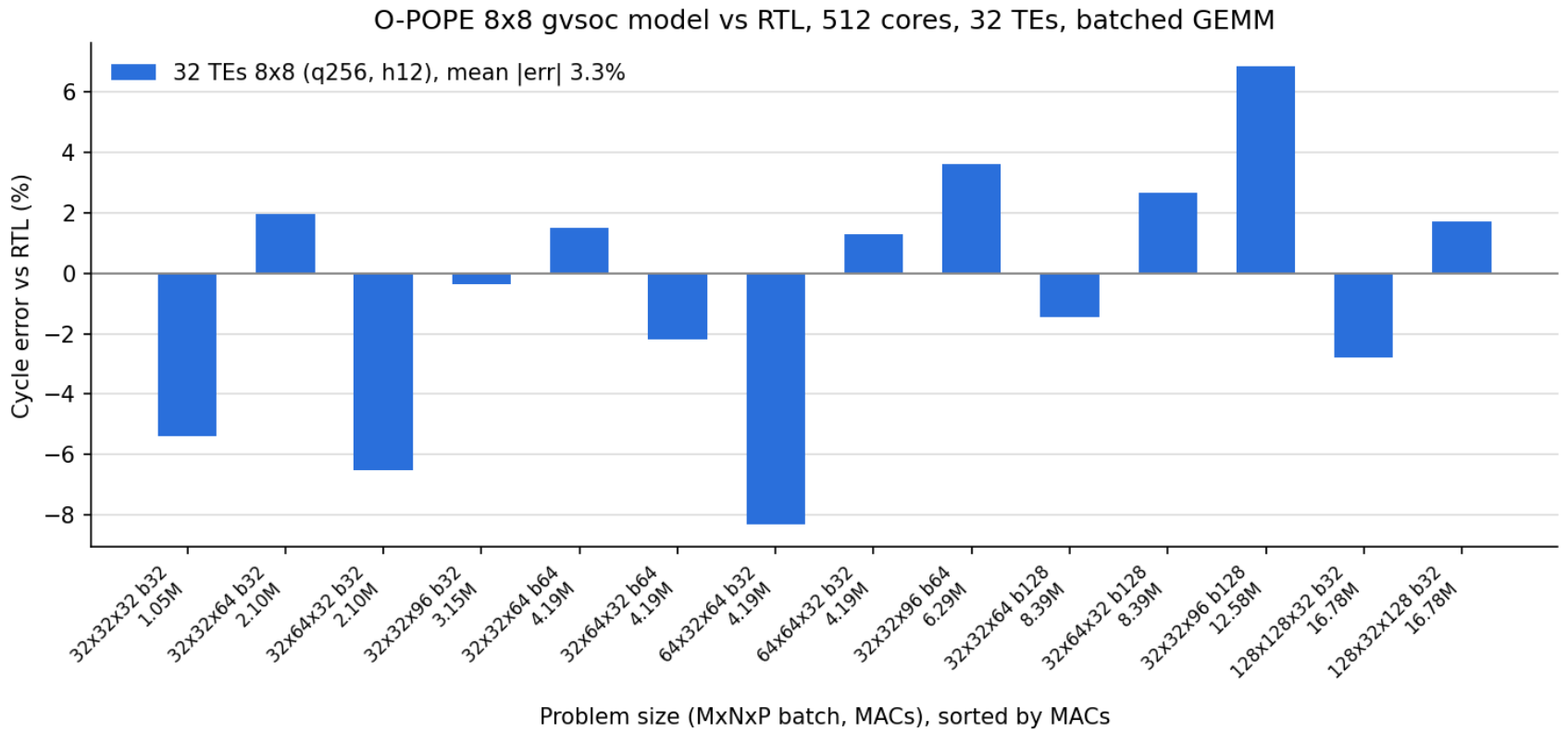


Planned actions (25/09/26):

- Depth-first scheduling
- Test of different HW configurations (number of PEs, number of TEs)

Calibration of GVSoC model

- `queue_depth` parameter, indicates the number of outstanding transactions
- `bank_hold` parameter, models bank conflicts



Depth-First Scheduling

- Depth first scheduling with overlapped PE and TE computation

```
for(i = 0; i < Batch; i++) {
    // im2col PE
    // convolution TE
}
```

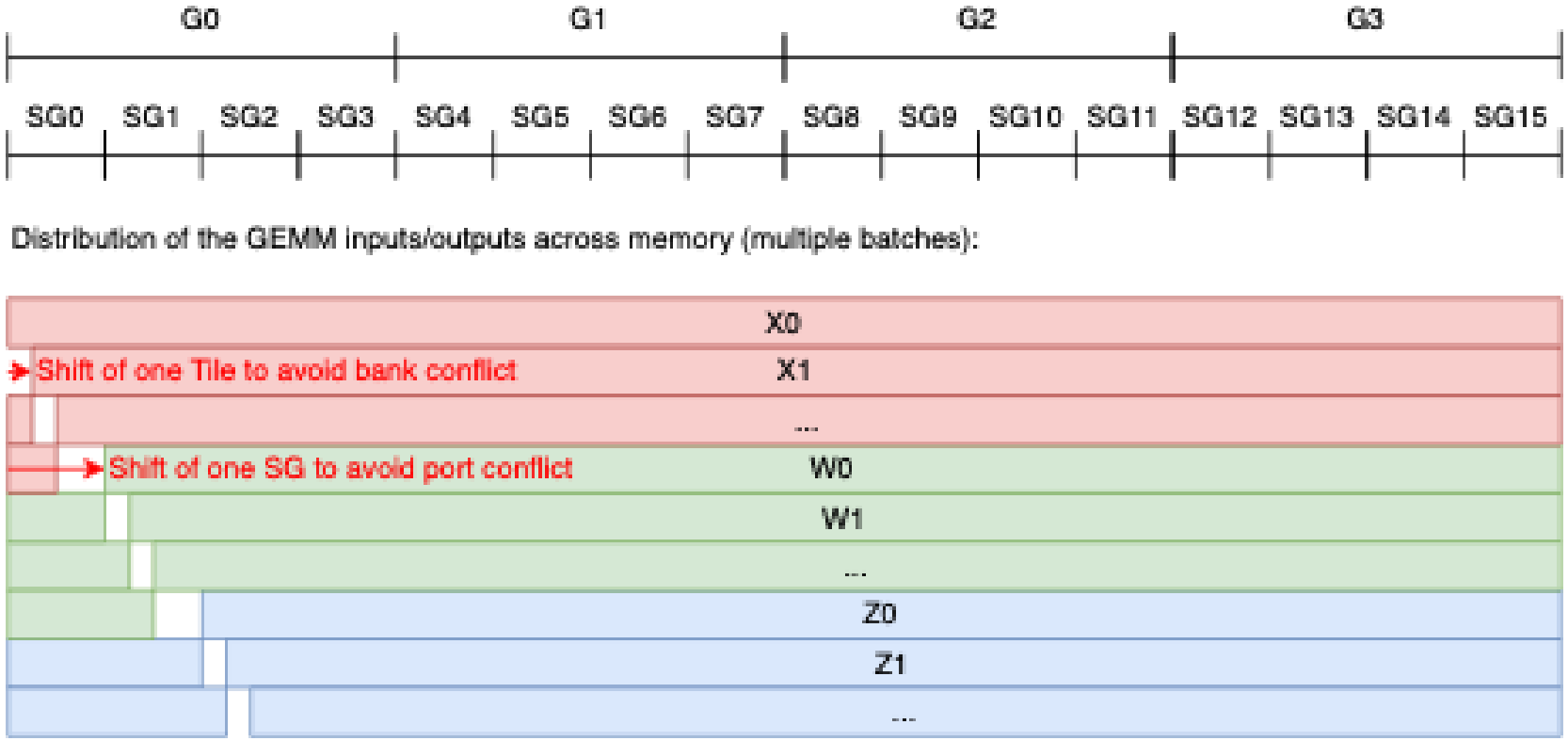


```
// im2col[0] PE
for(i = 1; i < Batch-1; i++) {
    // convolution[i-1] TE
    // im2col[i] PE
}
// convolution[Batch-1] TE
```

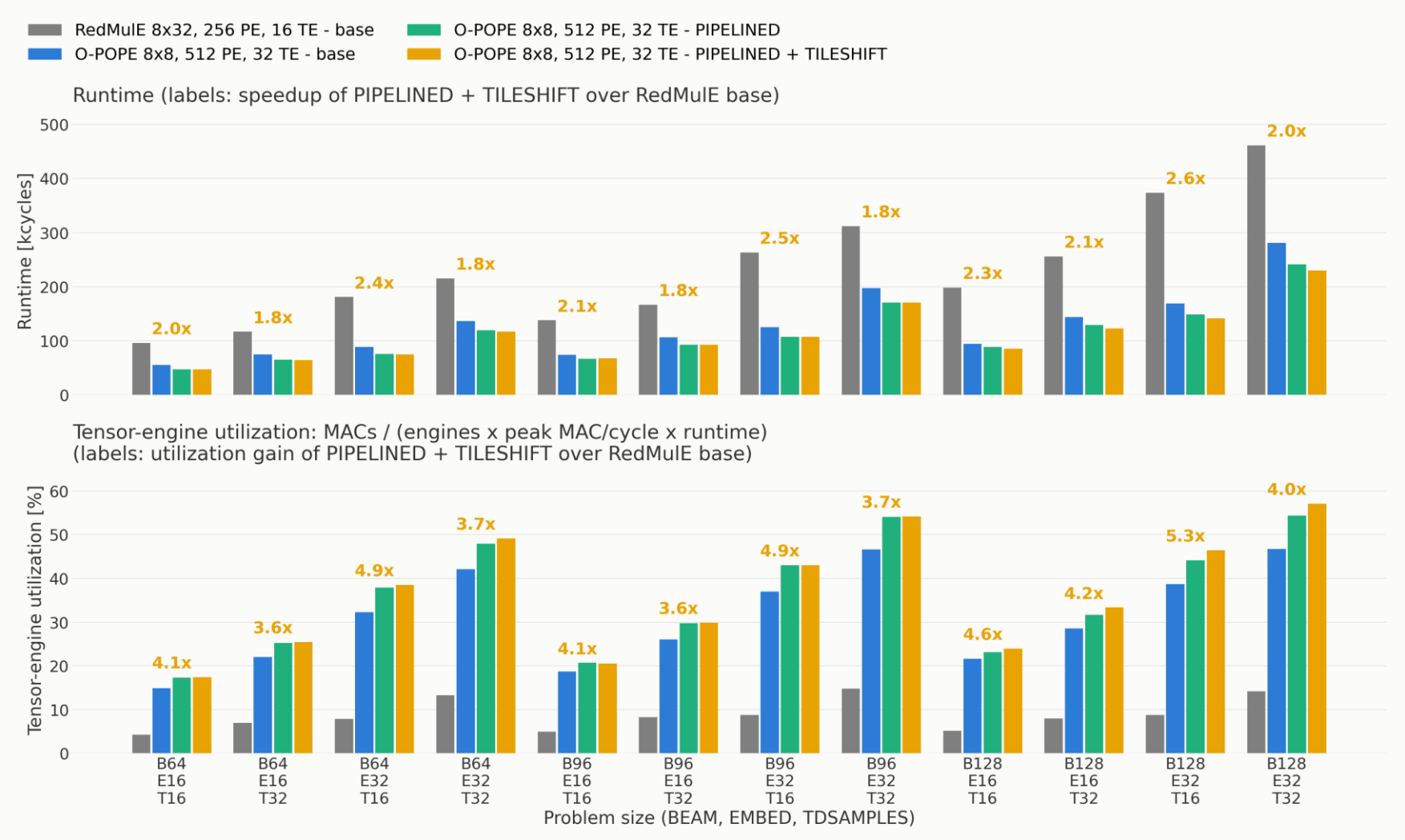
- With multiple TEs the Batch loop is divided between TEs
- With multiple Clusters the Batch loop is divided between TEs and Clusters

Matmul data allocation

- We shift the independent problems by `NUM_BANKS_PER_TILE` to avoid bank conflict
- We shift X, W, Z by `NUM_BANKS_PER_SUBGROUP` to avoid port conflicts



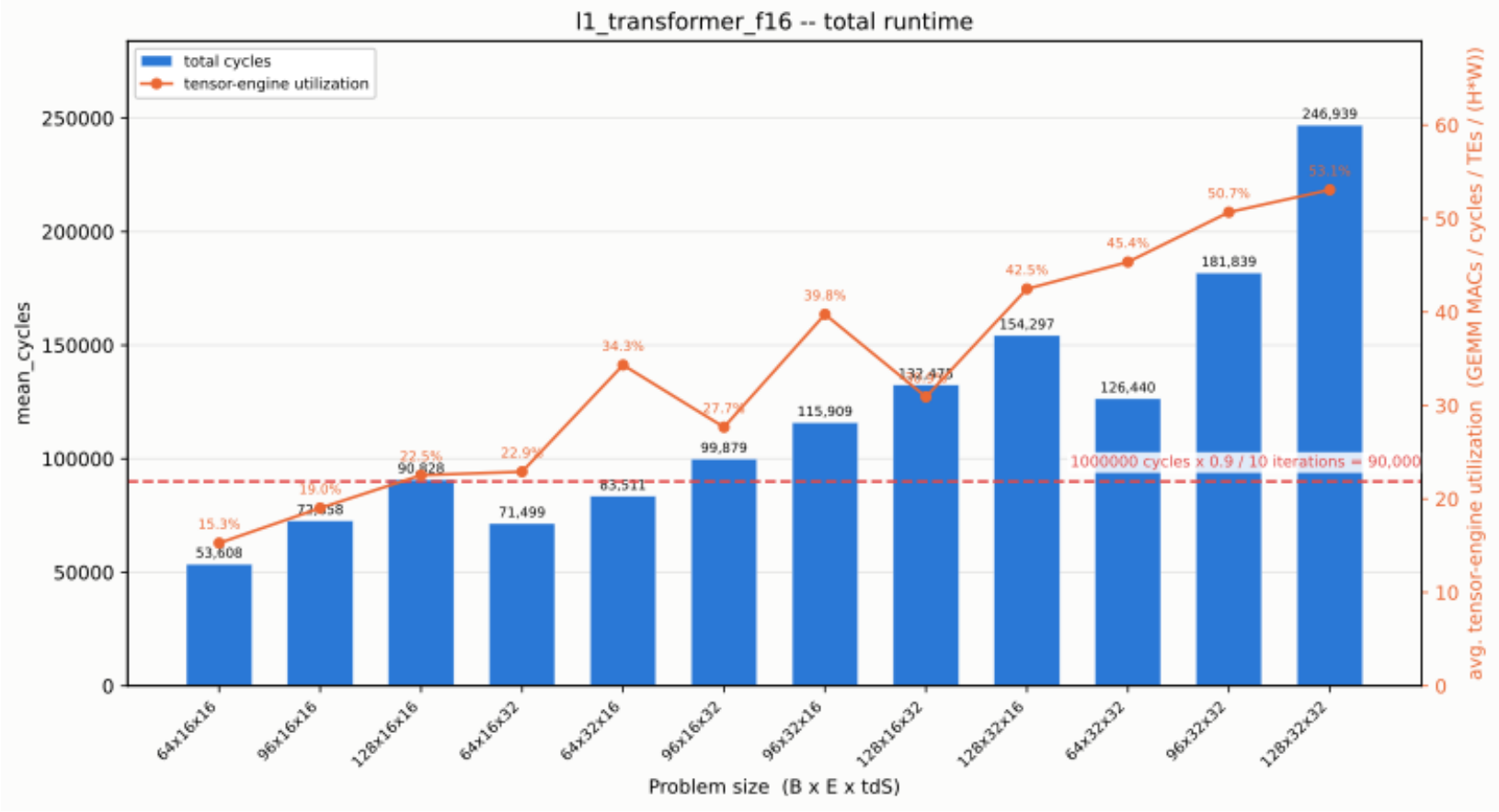
Utilization improvement



4x more utilization and 2x less runtime = many small TEs performs better than few large TEs

Single-Cluster Execution

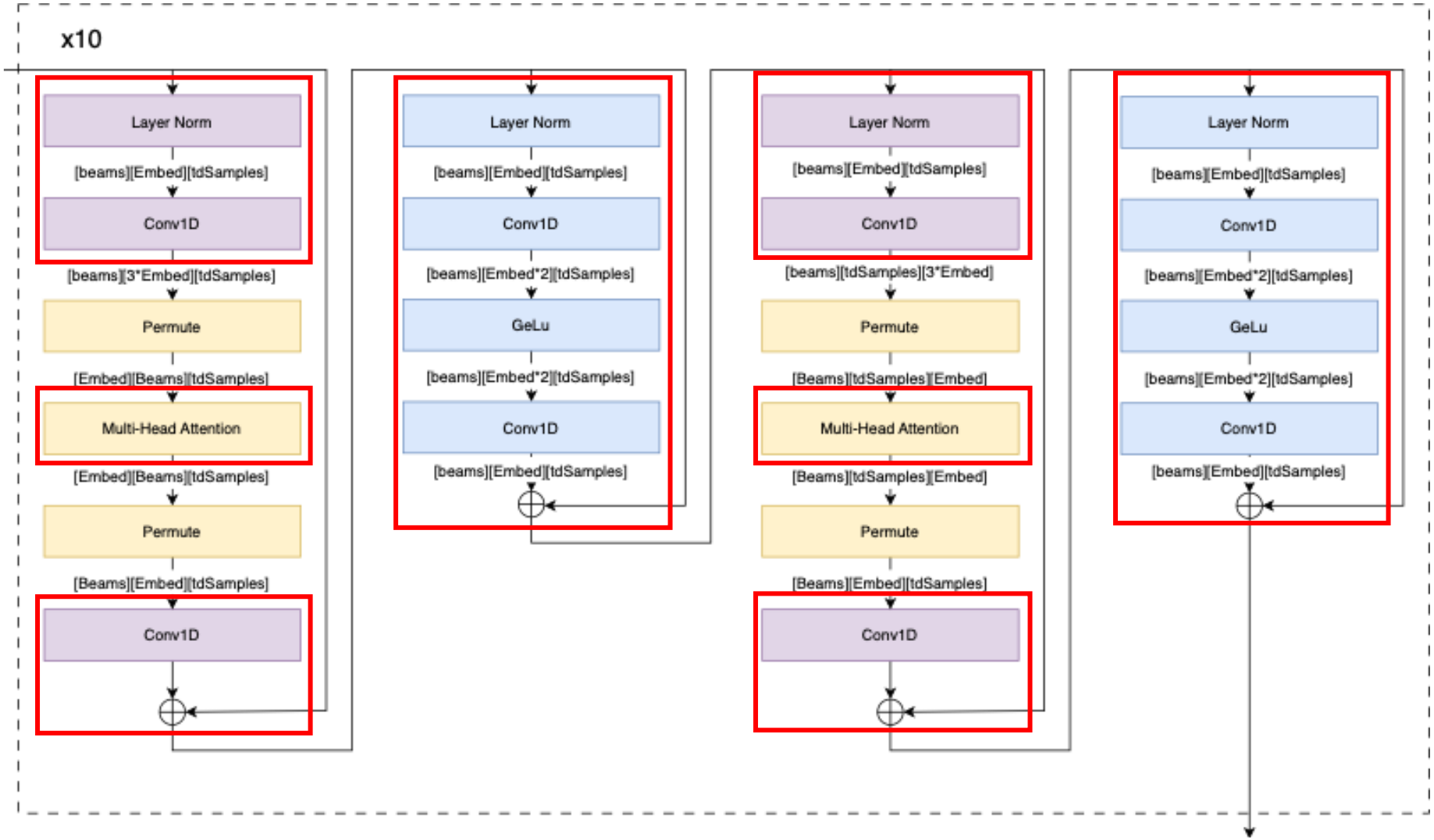
- 32 8x8 O-POPEs, 512 PEs ➔ peak 53% utilization
- 6-10 iterations in 9 out of 12 input problem sizes



Parallel implementation

Parallel over N clusters

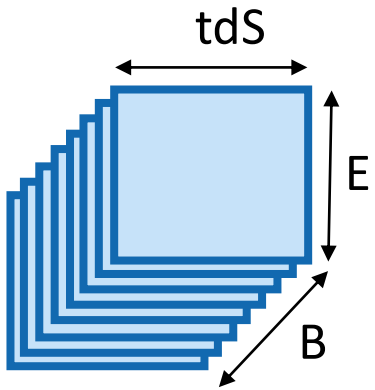
Parallel over N clusters

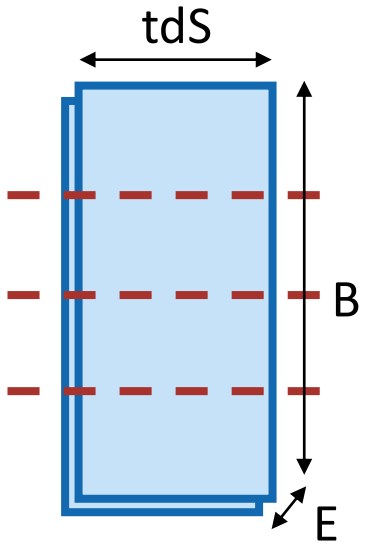


Parallel implementation

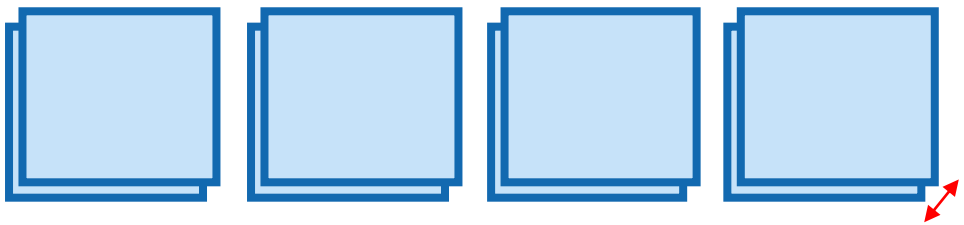
Parallel over N clusters Embed/tdSamples < Beam:

- Batch parallelism for convolutional layers
- Batch + Tensor parallelism for attention layers



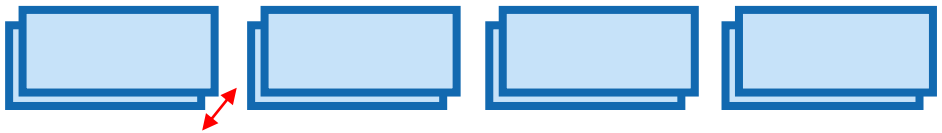


Parallel over Clusters



Parallel over TEs

Parallel over Clusters

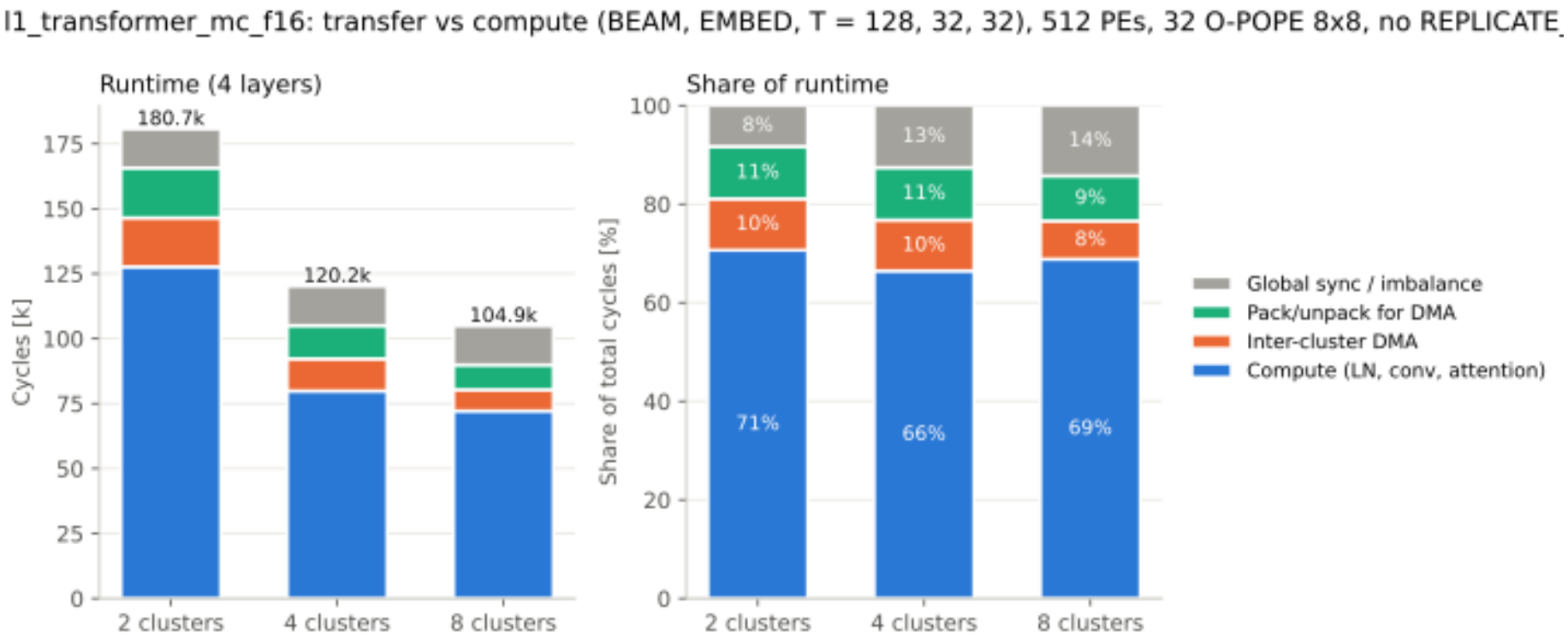


Parallel over TEs

Multi-Cluster implementation

Parallelization achieves the required runtime

- We lose utilization on absence of overlap (batch divides between clusters)
- We lose utilization on cluster-to-cluster data transfers and synchronization



Further actions

- More tensor parallelism for depth-first scheduling also on the single-cluster
- The softmax step is PE limited on the single-cluster (because problem sizes do not help us, 128x32x128 MATMUL gives 128x128 matrix) ➔ Try out configurations with more PEs

3DIC: What are issues we want to solve through 3D?

- **TensorPool is an interconnect-driven design**
  - High BW Xbars but large die area overhead

Interco-Level	No. Interco/Cluster	Each Configuration	Round Latency (Cycles)	Range (Cost)
Local Tile	64 x 2 (req/resp)	stream_xbar (4+3)x32	1	Intra-tile (low)
Local SubGroup	16	xbar 4x4	3	Intra-SG (low)
Inter-SubGroup	48	xbar 4x4	5	Intra-Group (mid)
Inter-Group	12	xbar 16x16	9	Full die span (high)
Cluster	0	XOR permutation Only	Unchanged	Full die span (high)

SubGroup design (physical) was good enough:
base-block, flatten, dense, no area waste, signoff

Here are costs:

1. Routing channel added for routing; Waste die area
2. Spill registers added for cutting physical wiring length

3D Exploration Plan

- **Plan A: SubGroup/Tile-level folding**
  - Mem-on-Logic (core+interco <-> L1 memory banks)
  - Done (MemPool-3D)
- **Plan B: Group-level folding**
  - Groups-on-Groups
  - Done (TensorPool-3D)
- **Plan C: Interconnect-centric partition**
  - Interconnect-on-Compute&Memory

	Plan A	Plan B	Plan C
Align existing RTL hierarchy	None, hierarchical module broken	exact	Xbars had to be extract
V-ports counts	162-552k	121k	121k
Additional pipeline	Needed	no	**Two removed**
Heat Diss.	Through SRAM Tier	Hard	**Better**
3D Partitioning Complex	Hard: logic folding in module (Tile) + Hier3D EDA Assembling	low	moderate
Routing overhead in #1 Tier	High, channel btw Groups still exsit	Reduced channel between Groups	**Closed channel between Groups**
Area util. in #2 Tier	More room for L1	Same as #1	**May low util.**, but: 1. Room for extra SRAMs (L2 Buffer) 2. Move intra-SG interco to this Tier

3D Interface (Hybrid Bounding Terminals) Analysis

- **39% area reduction (For a Group)**
- **At least 120752 ports need to go vertically**
  - 30197/Group (from RTL)

	2D	3D, implemented
SubGroup	952.3 µm square	952.3 µm square
Group channel	0.55 x SubGroup pitch	**100.0 µm fixed**
Group	2432.3 µm square, 5.92 mm²	**2008.5 µm square, 4.03 mm²**
Cluster channel	0.18 x Group pitch	**none, Groups abut**
Cluster / compute die	5306.2 µm square, 26.50 mm²	**4020.8 µm square, 16.17 mm²**

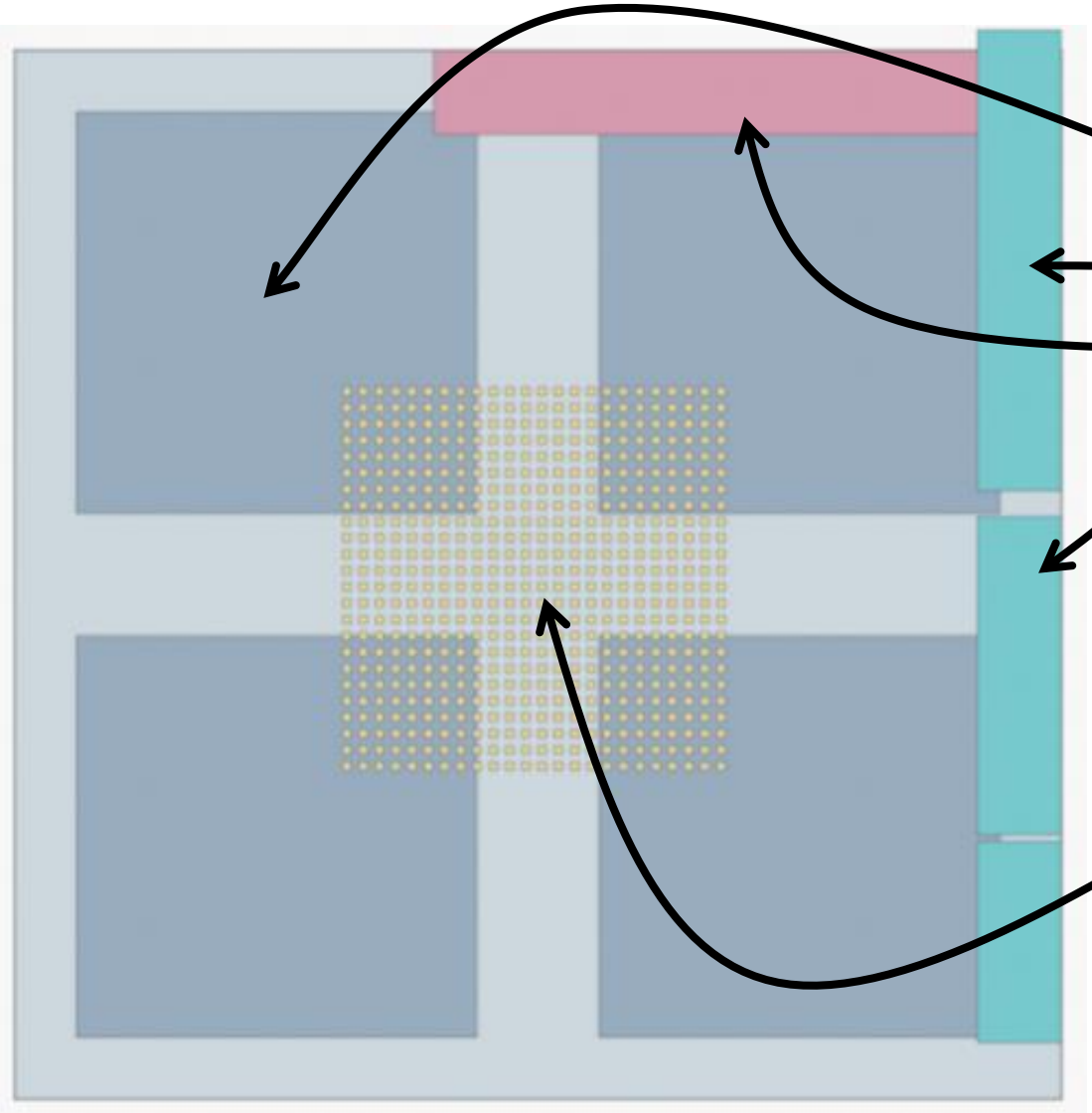
Bus family	Per Group	Total	Share
`tcdm_master_req` 108 + 2	5,280	21,120	17.5%
`tcdm_master_resp` 144 + 2	7,008	28,032	23.2%
`tcdm_slave_req` 108 + 2	5,280	21,120	17.5%
`tcdm_slave_resp` 148 + 2	7,200	28,800	23.8%
TCDM subtotal — 48 channels x 516 bits	**24,768**	**99,072**	**82.0%**
`axi_mst_req` 717 x 4 masters	2,868	11,472	9.5%
`axi_mst_resp` 528 x 4 masters	2,112	8,448	7.0%
DMA request, ready, metadata	117	468	0.4%
`wake_up` 64 cores	64	256	0.2%
`ro_cache_ctrl` 258	258	1,032	0.9%
Per Group	**30,187**	**120,748**	
Clock, reset, test mode, scan enable	—	4	
Total, measured		**120,752**	**100%**

3D Interface (Hybrid Bounding Terminals) Analysis

- **In TSMC7:**
  - AP.W.1 and AP.S.1 are both 1.8um
  - Pitch floor this tech supports is 3.6um in a REAL chip
- **With current ports number, the loosest pitch that still fits is 8.14um**
  - We also need rooms for PG, dummy
  - 4.50um is a good one (Now we talk about the REAL tech. supports)

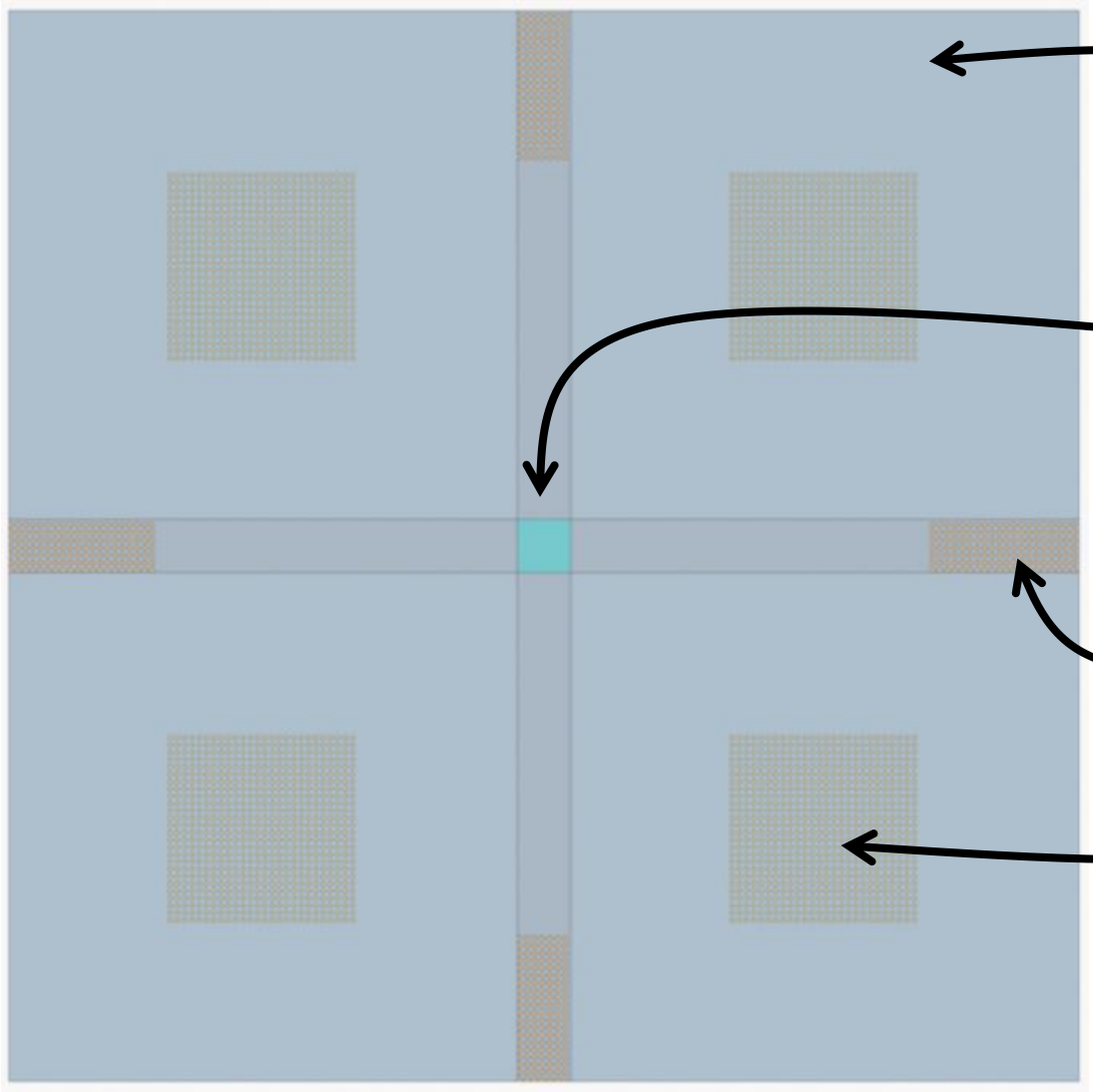
Pitch	Pad	Sites per Group	Signal sites	Utilisation	Verdict
1.00 µm	0.50	2006 x 2006	2,012,018	1.5%	below the AP floor, needs a bonding level
2.00 µm	1.00	1002 x 1002	502,002	6.0%	below the AP floor, needs a bonding level
3.60 µm	1.80	555 x 555	154,012	19.6%	fits, at the PDK floor
4.50 µm	**1.80**	**444 x 444**	**98,568**	**30.6%**	**fits — implemented**
9.00 µm	1.80	221 x 221	24,420	123.6%	**does not fit**

3D Design View: SubGroup



- Computing TensorTile (Flatten)
- Interfaces to crossbars for Inter-SG connection (2D) + AXI/DMA
- Vertical interfaces:
  - To crossbars for cross-Groups connections
  - CLK, control signals propagation

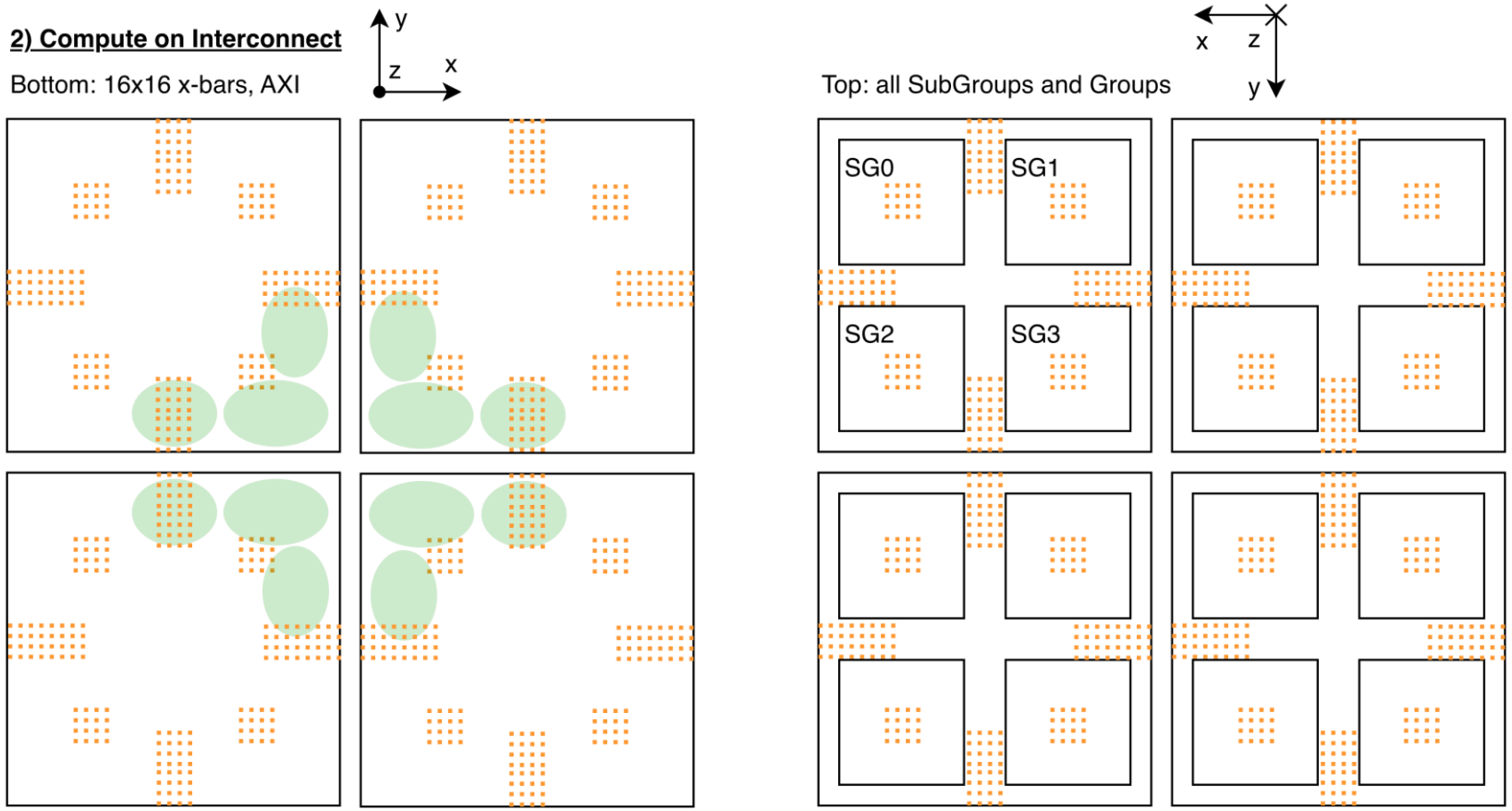
3D Design View: Group



- 4 SubGroups
- Routing channel with wiring cross
  - Small channel for wiring only
- Vertical interfaces:
  - AXI & DMA
  - To crossbars for cross-Groups connections, CLK, control signals propagation

3D Status (Synopsys flow 3dic)

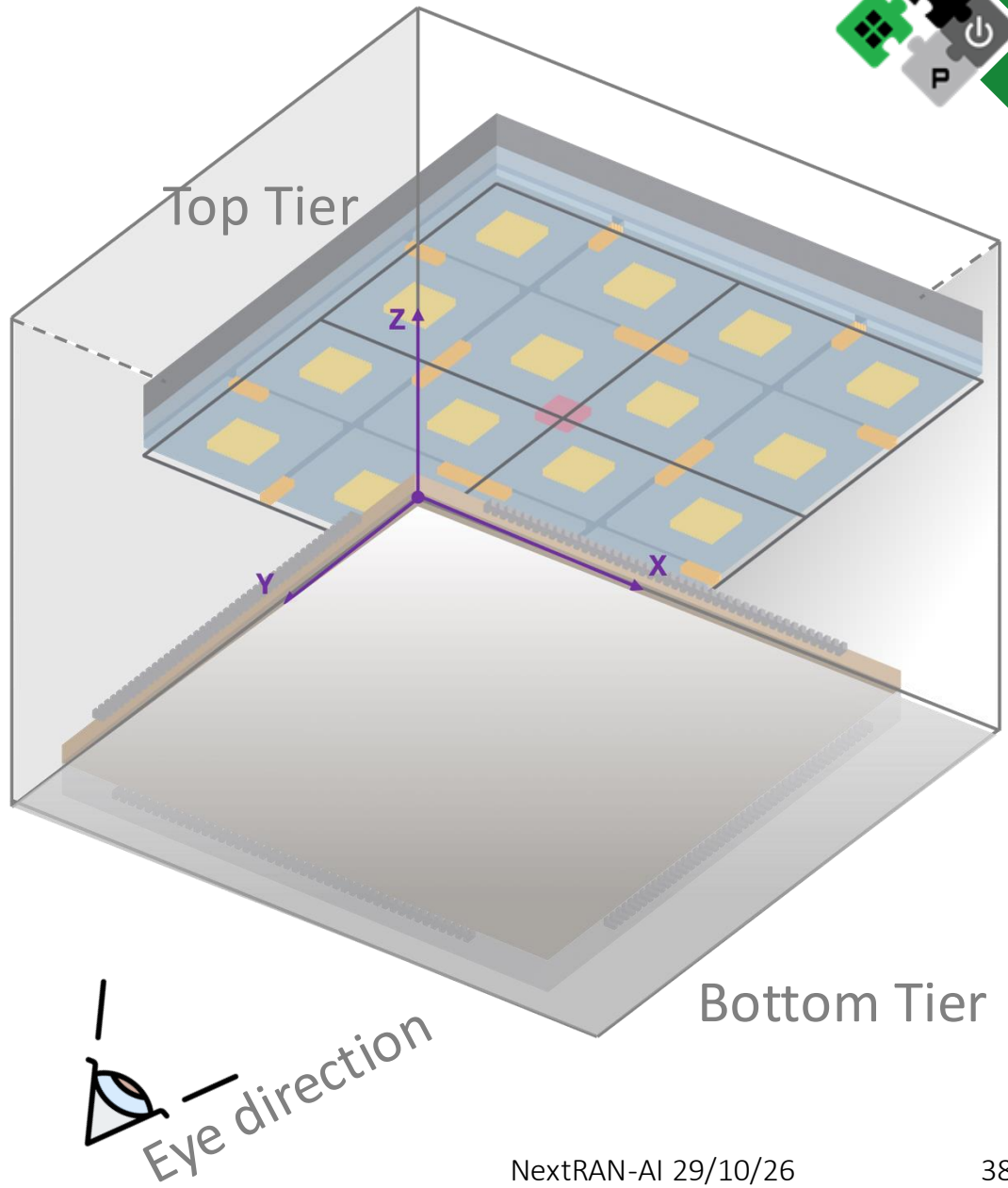
- **Interconnect-Tier is flattened design; Computing-Tier is hierarchical design.**



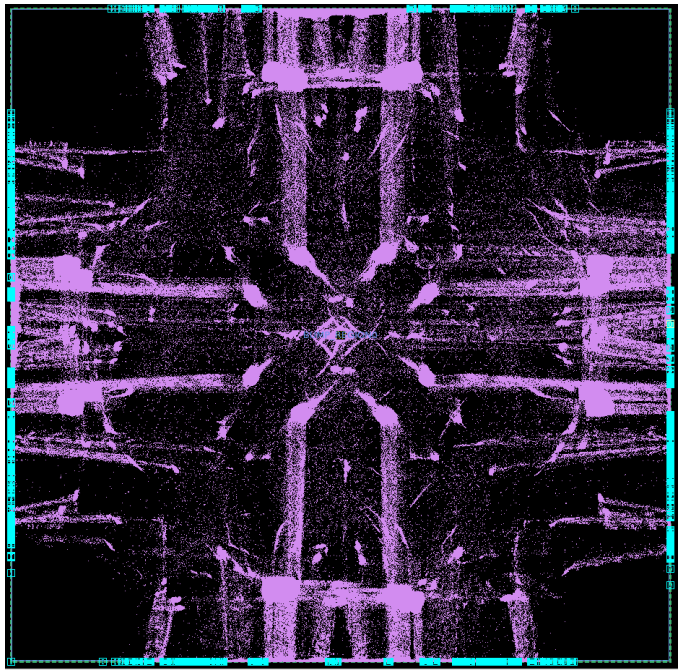
3D Design View: Tier Stacking

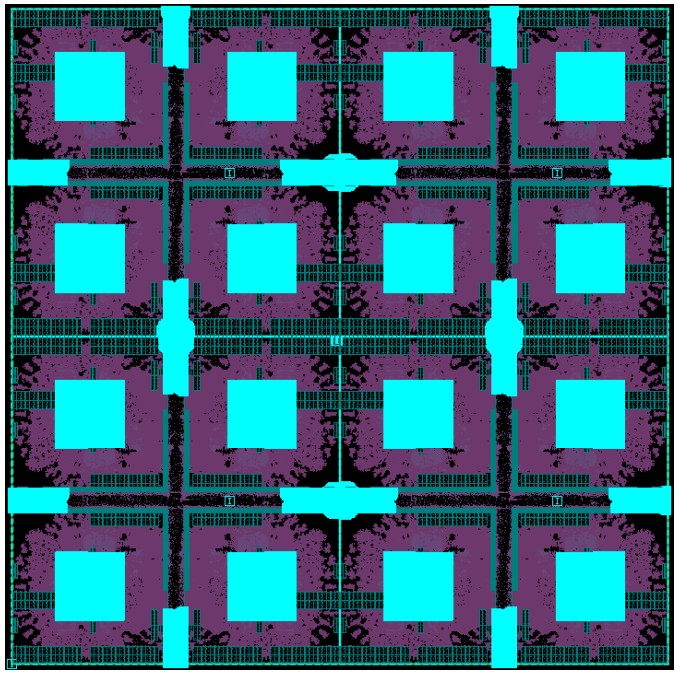
- 4 Groups (16 SGs) on Top-Tier
  - Memory and Computing logic on Top
  - Good for heat-diss.
  - Latency: 1-3-5-5
    - Four cycles less
    - remove 147840 cut registers from design

- 12 Crossbars, AXI, DMA etc. on Bottom-Tier
  - Low heat
  - Easy connection to packing substrate



Physical PnR Tier View





Dimension Breakdown		
	X (um)	Y (um)
SubGroup	920	920
Group	1945	1945
Comp Tier	3890	3890
Interco Tier	3890	3890

3D Folding	Clock Freq.	Tier area	TCDM latency	Heat Dissip.	Design Potential
Group-on-Interco	**1.05GHz (TT)**	15.13 mm^2	**1-3-5-5(7)**	**Good**	- Further reduce area (move SG xbars) - Larger interco. design (reduce Hier-level, BW++)
Group-on-Group	0.9GHz (TT)	11.47 mm^2	1-3-5-9	Norm	-

3DIC Milestones and Open Points

- **So far for 3D TensorPool exploration:**
  - Group-on-Group folding (done)
  - Group-on-Interco folding (today present)
    - Improvement and feasibility analysis
    - RTL: Hierarchical split & restructure (done, open-sourced)
    - 3D physical implementation (all flow steps implementation done)

- **Open point:**
  - Group-on-interco. folding:
    - Fine-tune the implementation now, final remote-Group latency will be confirmed.
    - Documentation in final report